Software Engineering

Course 3 – 10 March 2025

adiftene@info.uaic.ro

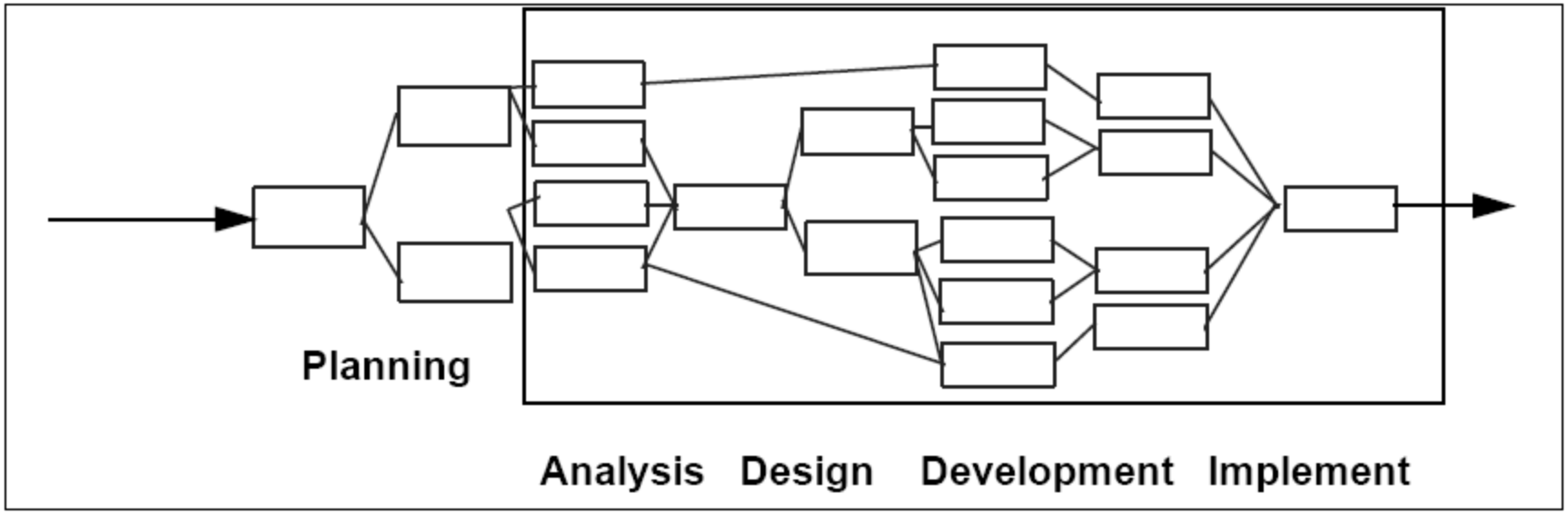
Content

- From Courses 1, 2…
- Modeling
- Modeling Languages
  - Graphic Languages
- UML – History
- UML – Definition
- UML – Diagram Types
- UML – Use Case Diagram
- UML – Class Diagrams

From Courses 1, 2

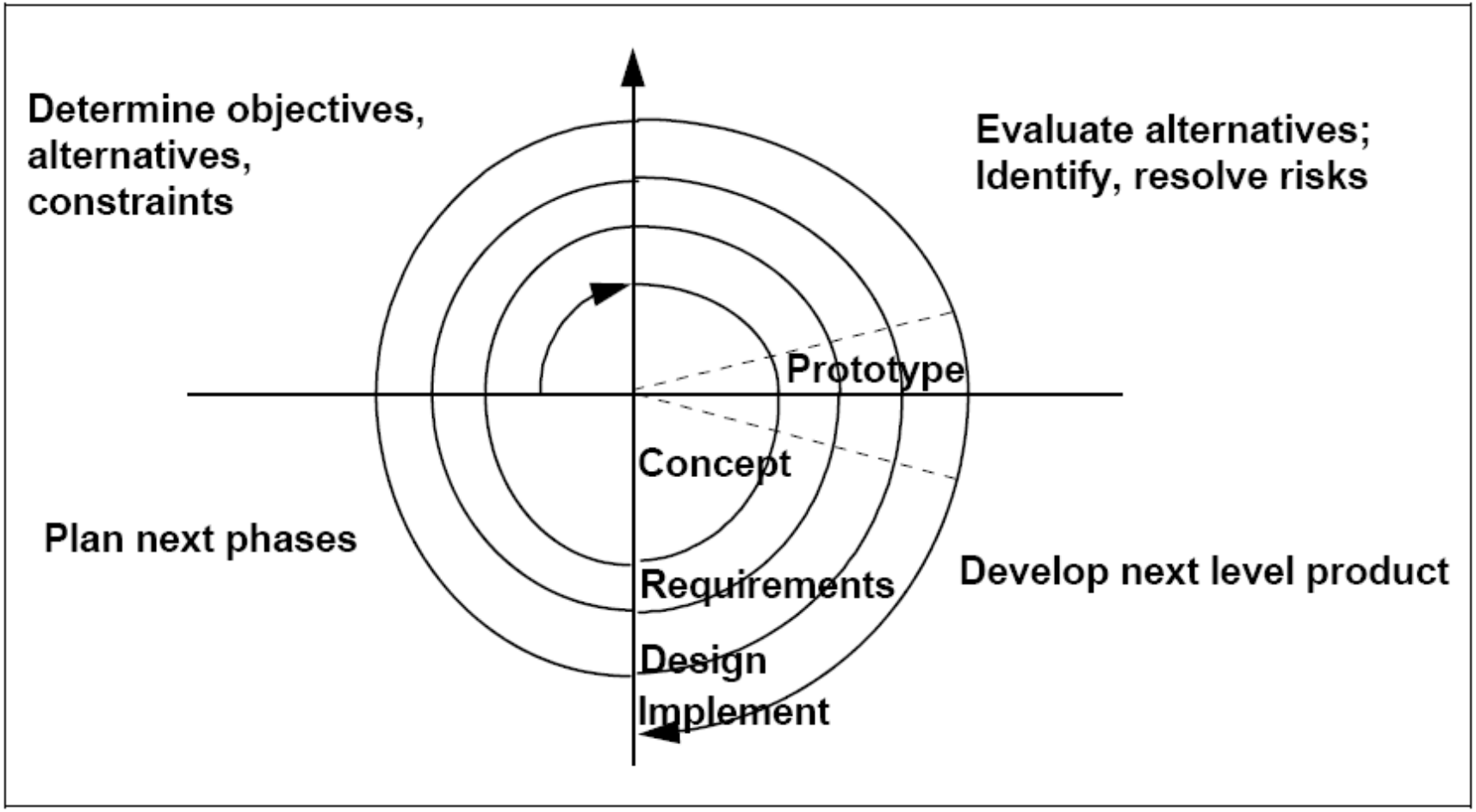
- Software engineering
- Phases for Developing Applications
- Developing Models
- Requirement Engineering

From Course 1



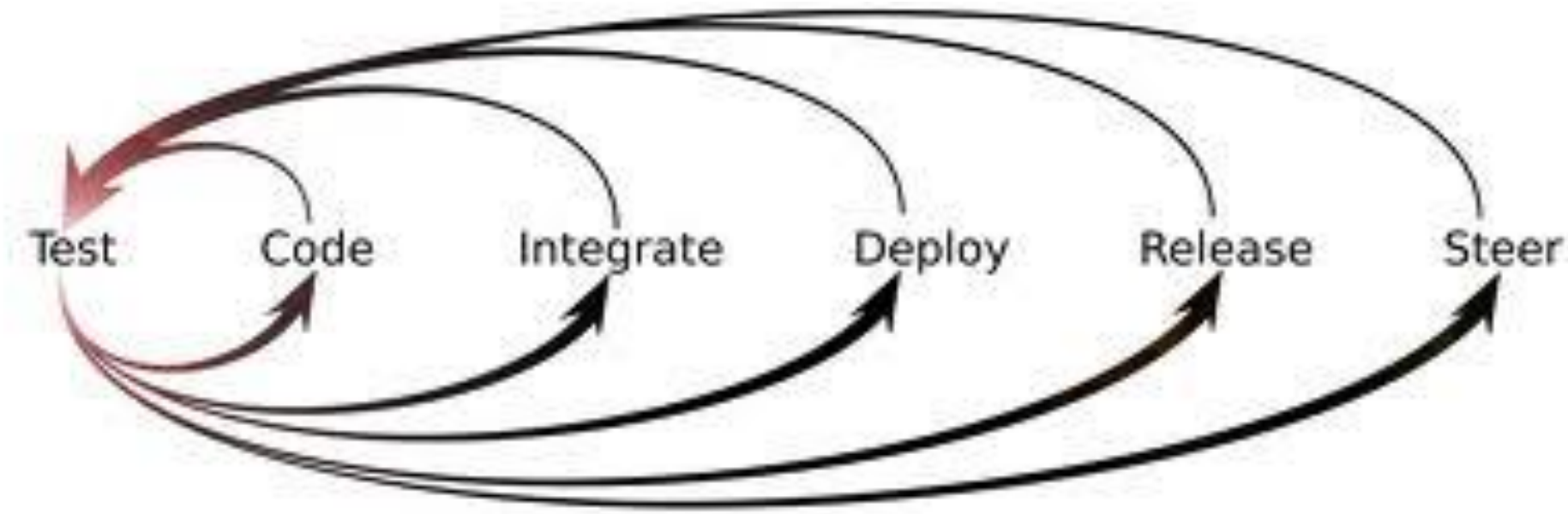
Waterfall model...

From Course 1



Spiral model

From Course 2

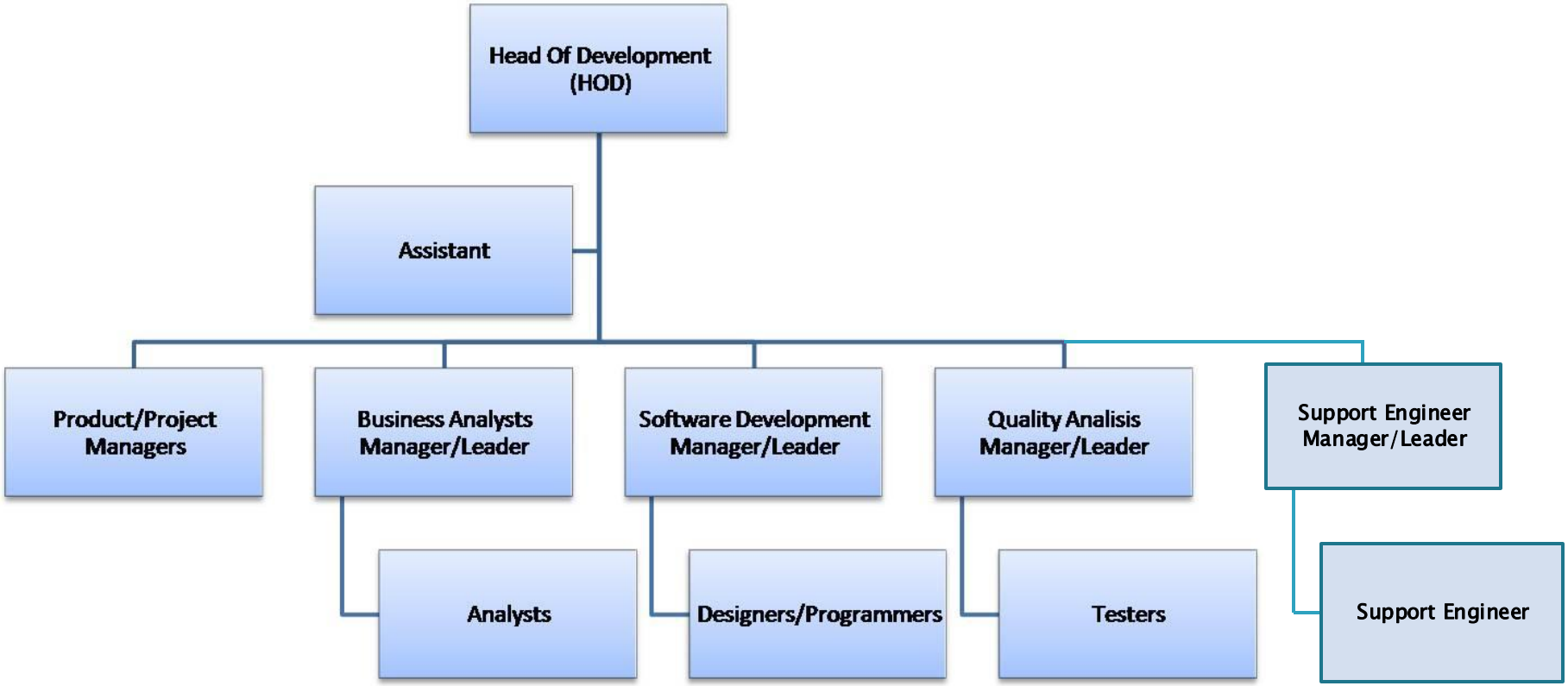


- XP, TDD…

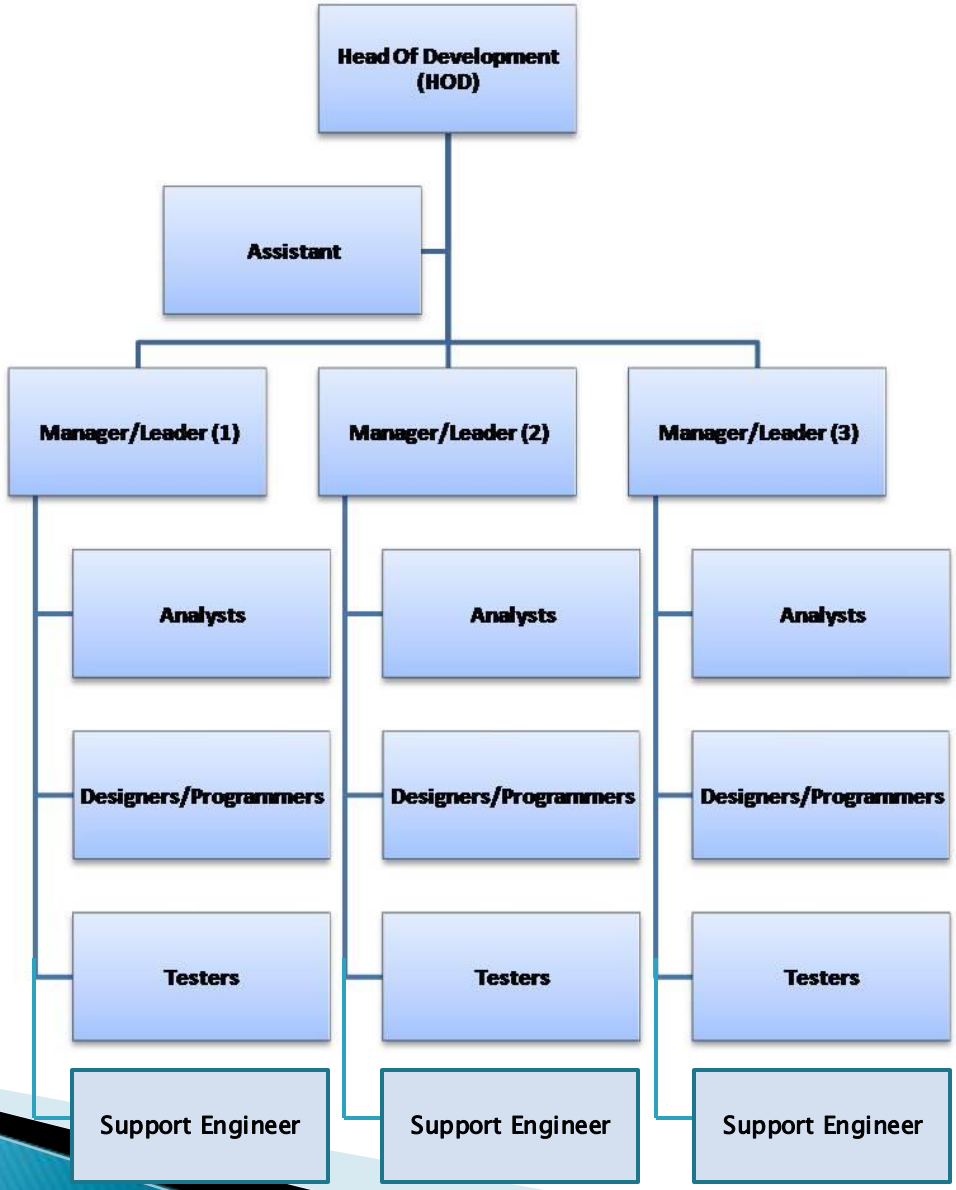
From course 2

- Requirements engineering:
  - Actors
  - Use cases

The hierarchy of a software company – Departments

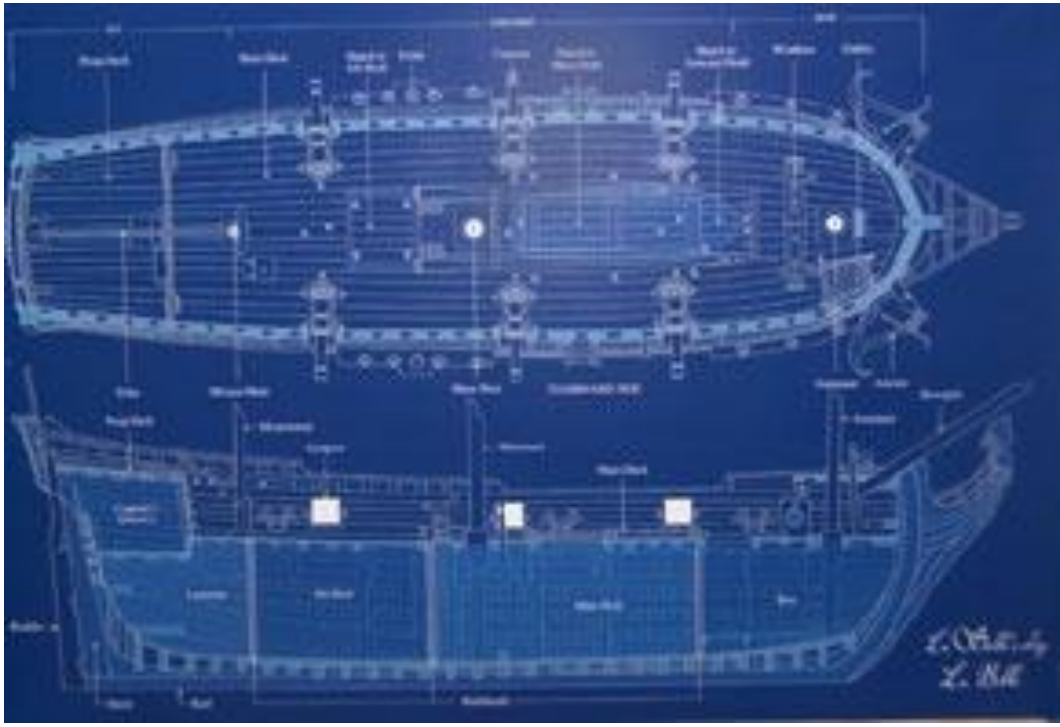


The hierarchy of a software company – Projects



Modeling – Why?

- What is a model?
  - simplification of reality
  - A detailed plan system (blueprints)



- Why do we model?
  - To understand better what we do
  - To focus on one issue at a time
- Where do we use modeling?

Modeling purposes

- Viewing a system
- Specifying its structure and / or behavior
- Providing a template to assist in building
- Documentation of decisions

Modeling Architecture

- Using **Use cases**: to present requirements
- With the help of **Design**: capture vocabulary and problem domain
- Using **Process**: capture processes and threads
- With the help of **Implementation**: we model the application
- Using **Deployment**: capturing the system from an engineering point of view

Modeling Principles

- Models influence the final solution
- You can use different levels of precision
- Good models have correspondent in reality
- One model is not enough

Modeling Languages

- The analysis and the design of a project must be done **before the implementation of the code**
- Currently, companies give special attention to this stage, since the **production and the reuse of software** depends on it
- **Modeling languages** were **created** for the analysis and design of programs
- A modeling language is an **artificial language** that can be used to express information or knowledge or systems

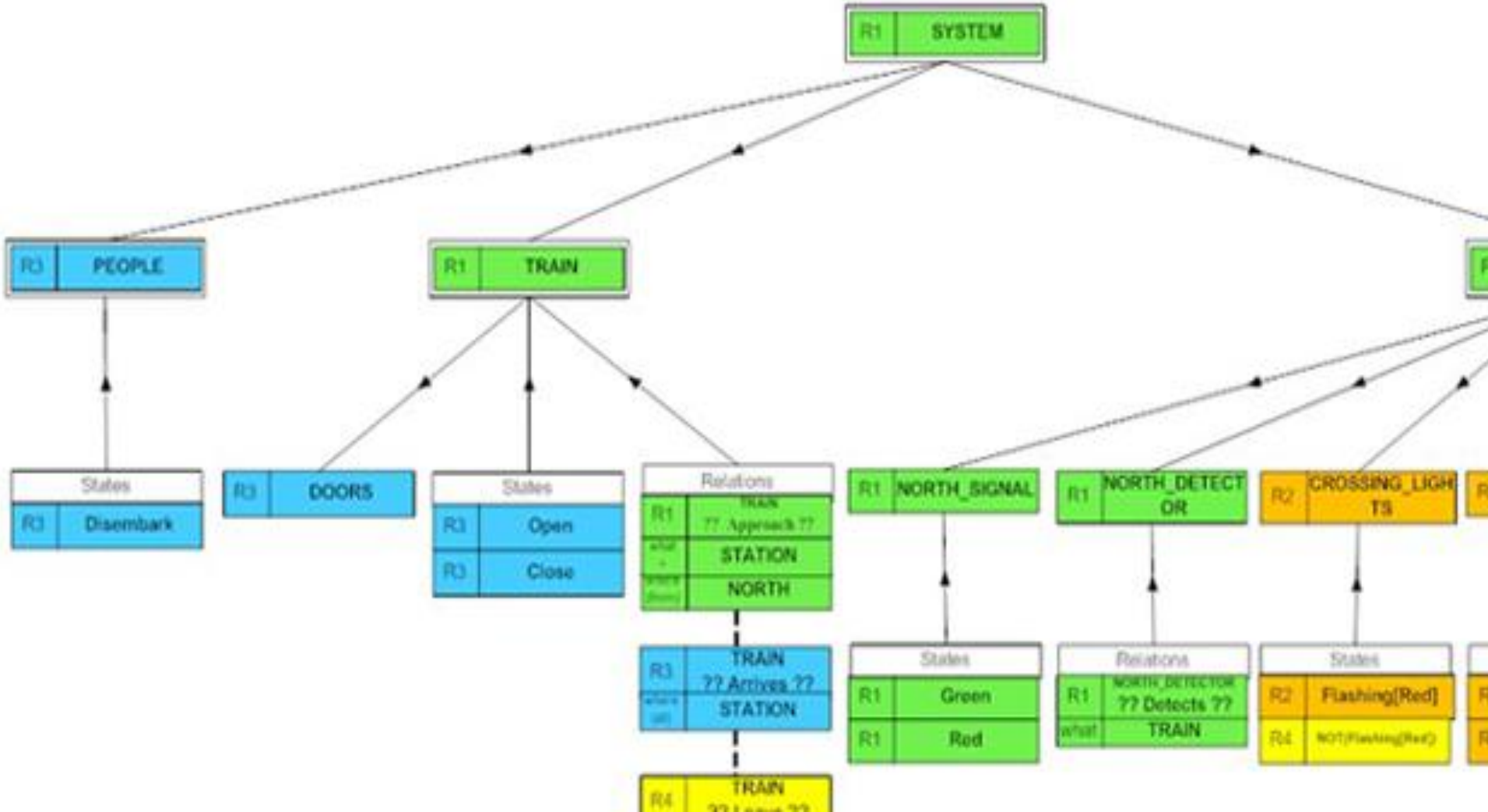
Modeling Languages – Types

- **Graphic languages**: behavioral trees, business modeling language, EXPRESS (data modeling), flowchart, ORM (roles modeling), Petri nets, **UML diagrams**

- **Specific languages**: algebraic modeling (AML), domain specific languages (DSL), architecture modeling (FSML), object modeling language, virtual reality modeling (VRML)

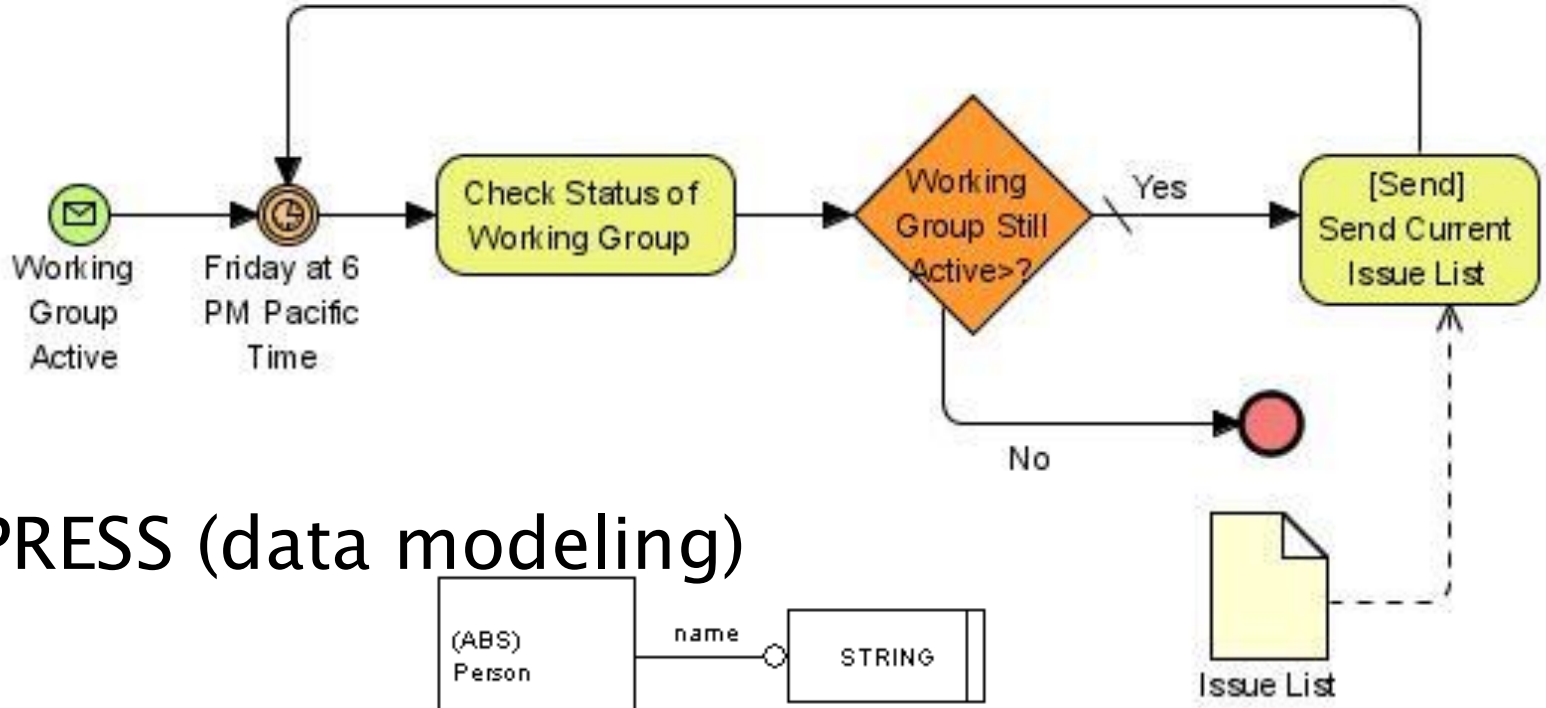
Graphic Languages 1

- Behavioral trees

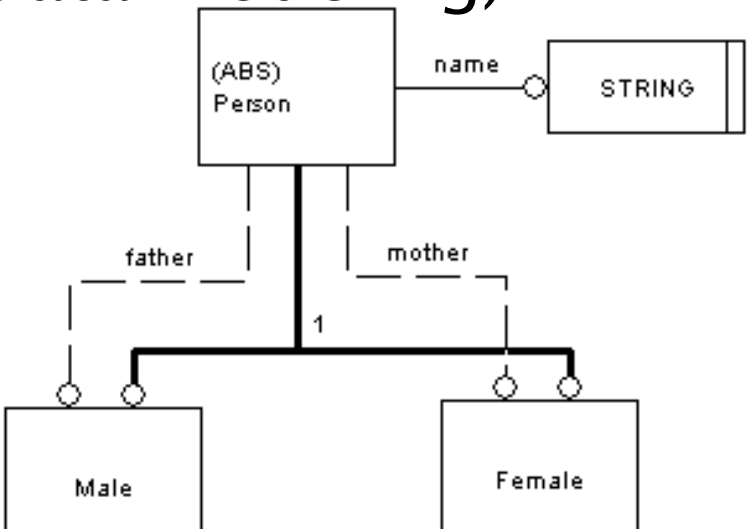


Graphic Languages 2

- Business processes

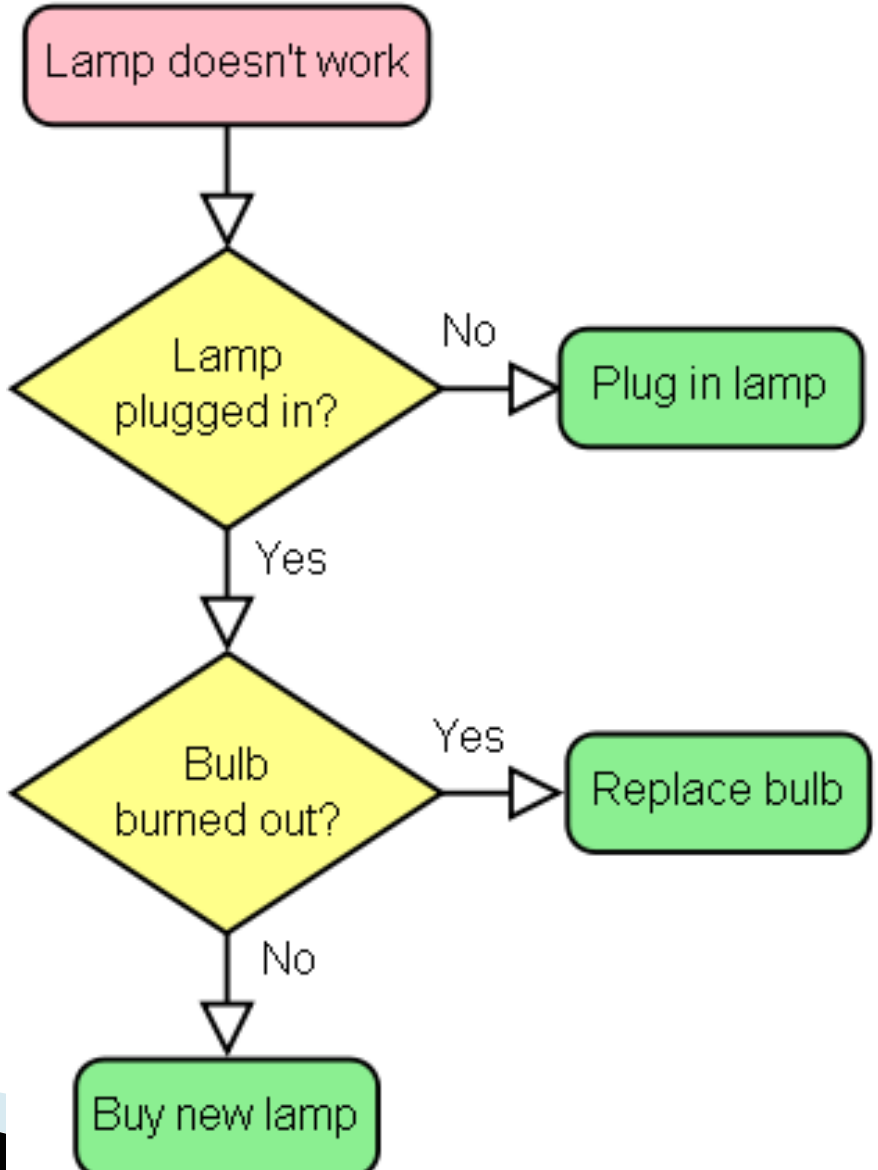


- EXPRESS (data modeling)



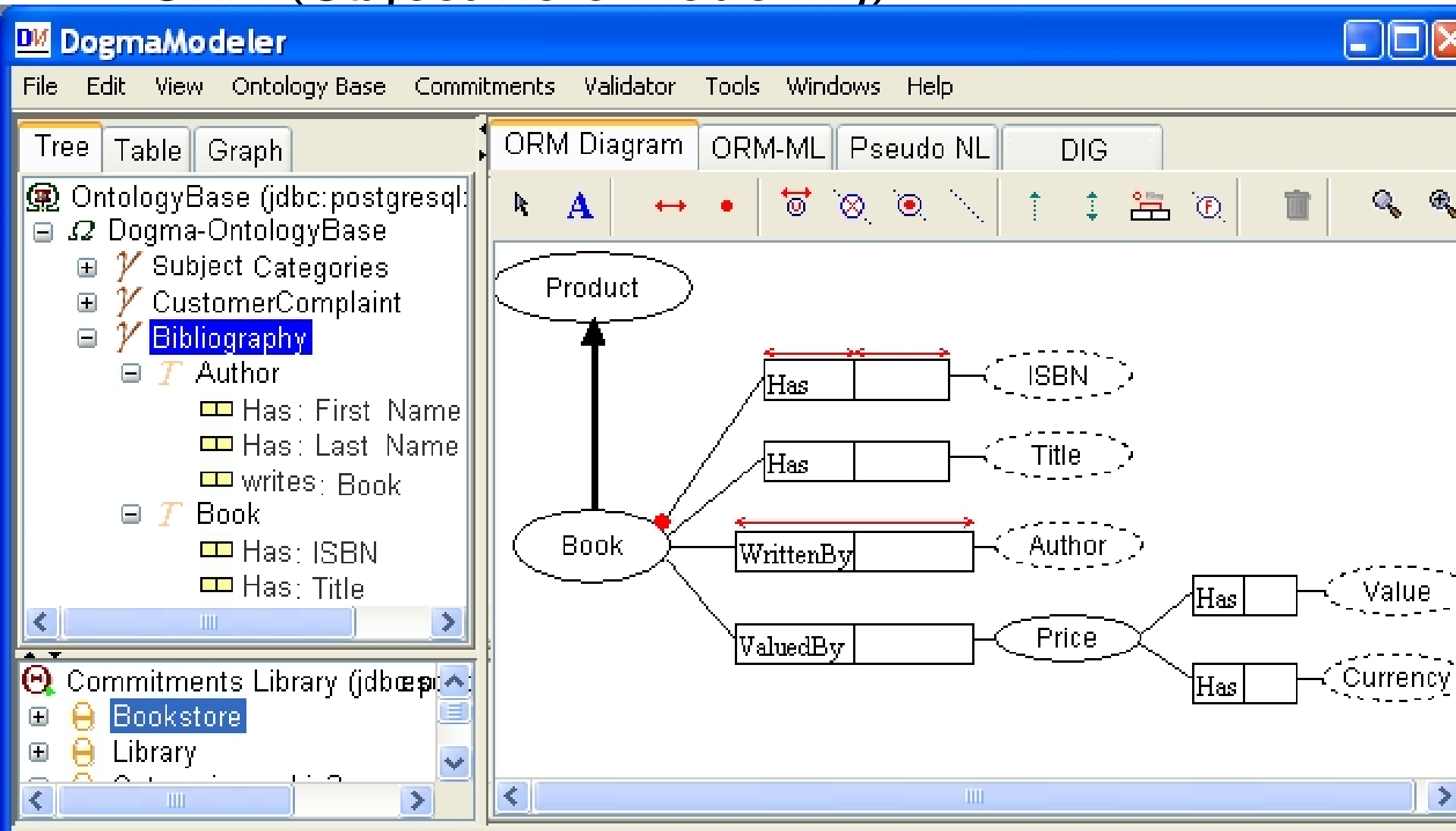
Graphic Languages 3

- Flowchart



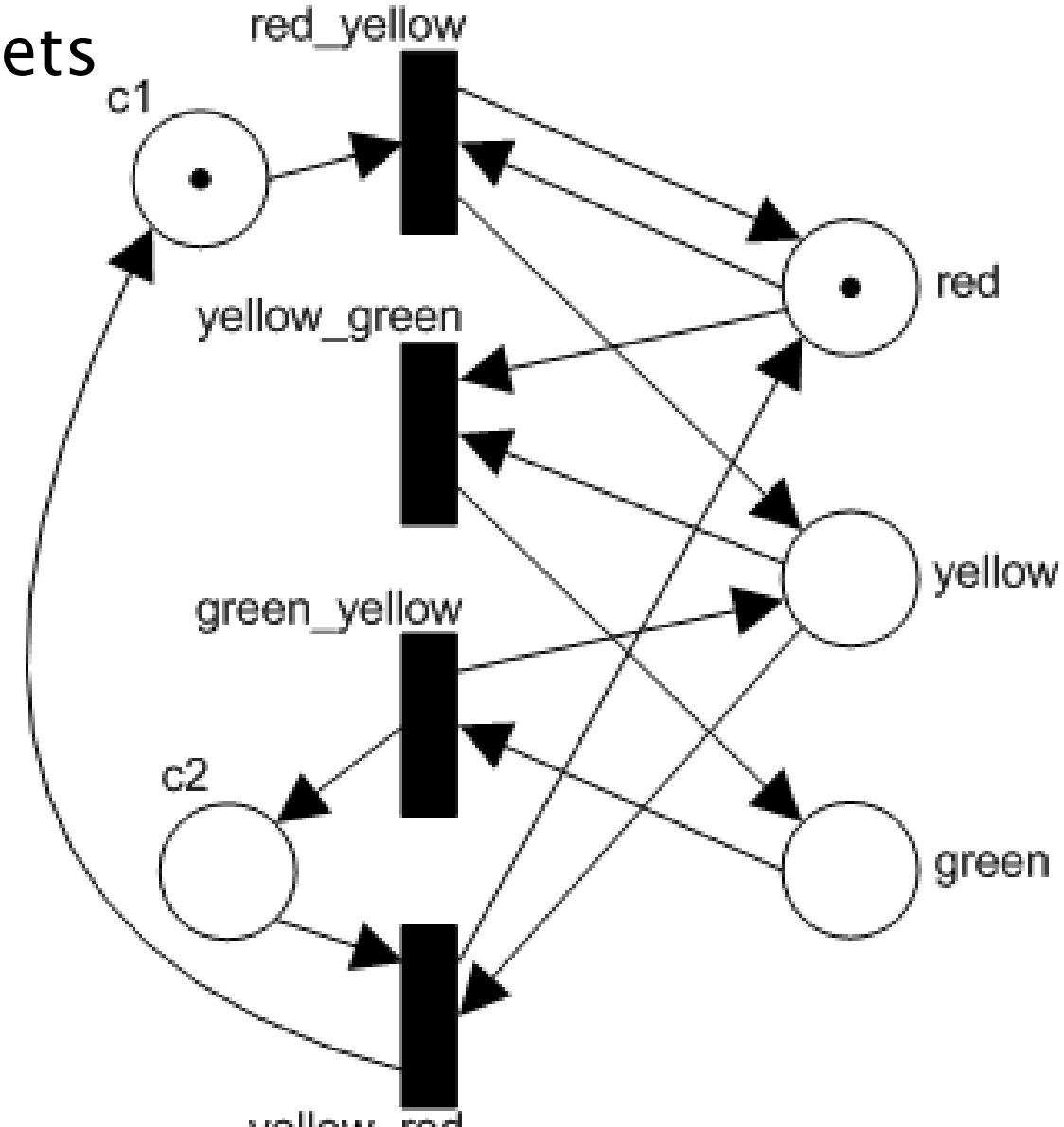
Graphic Languages 4

- ORM (Object Role Modeling)



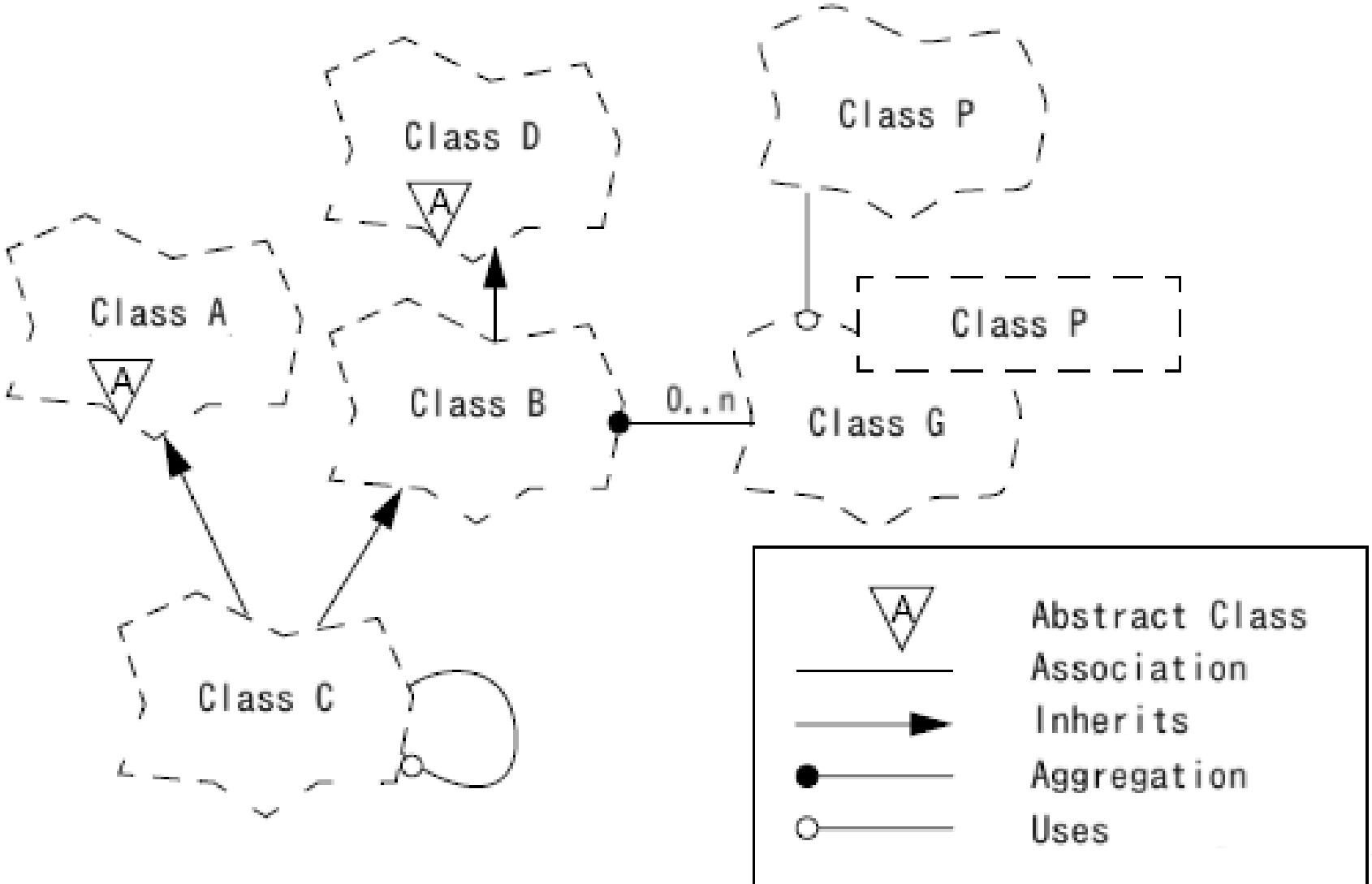
Graphic Languages 5

- Petri Nets



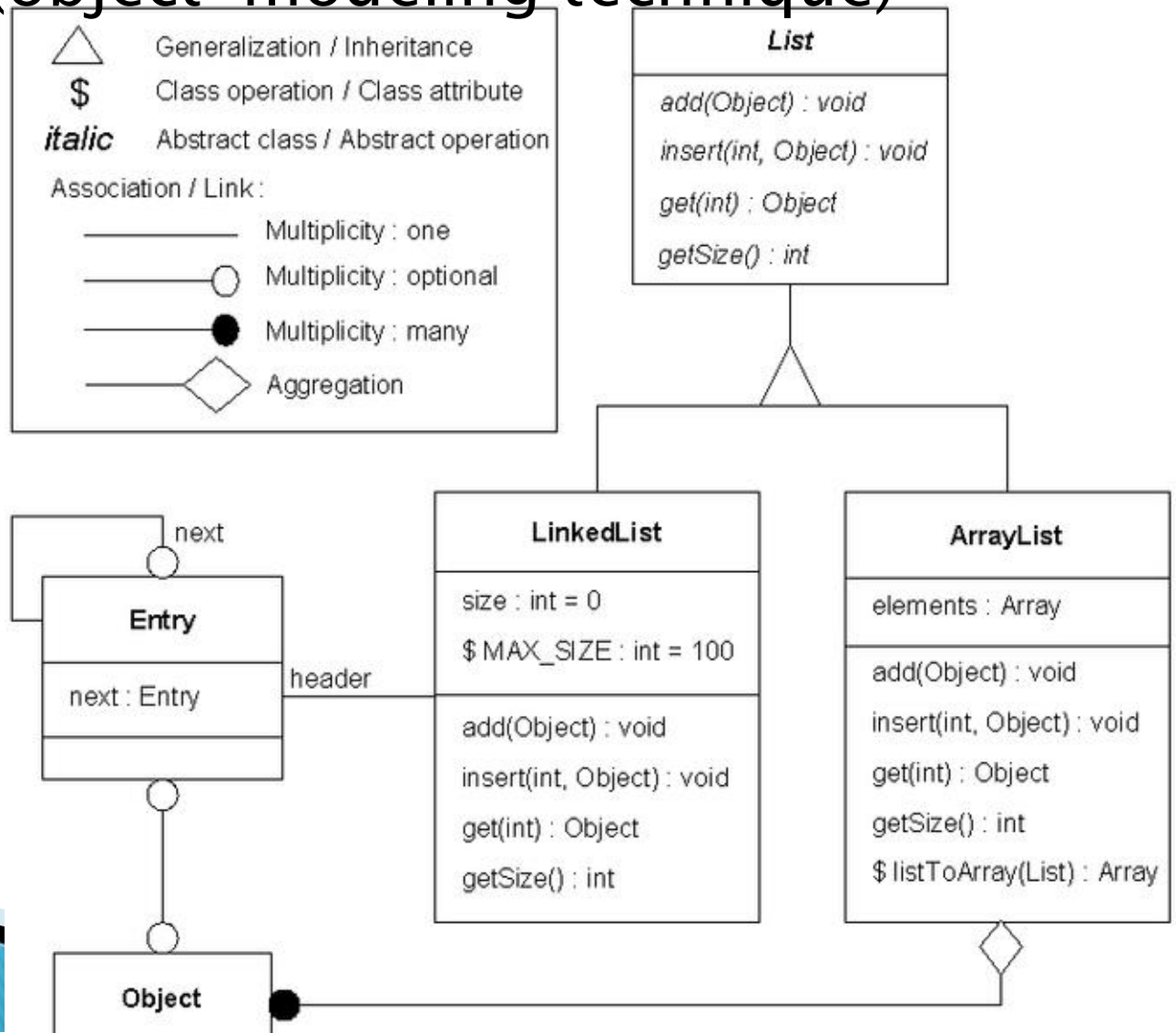
Graphic Languages 6

- Booch Method (Grady Booch) – analysis and oo design



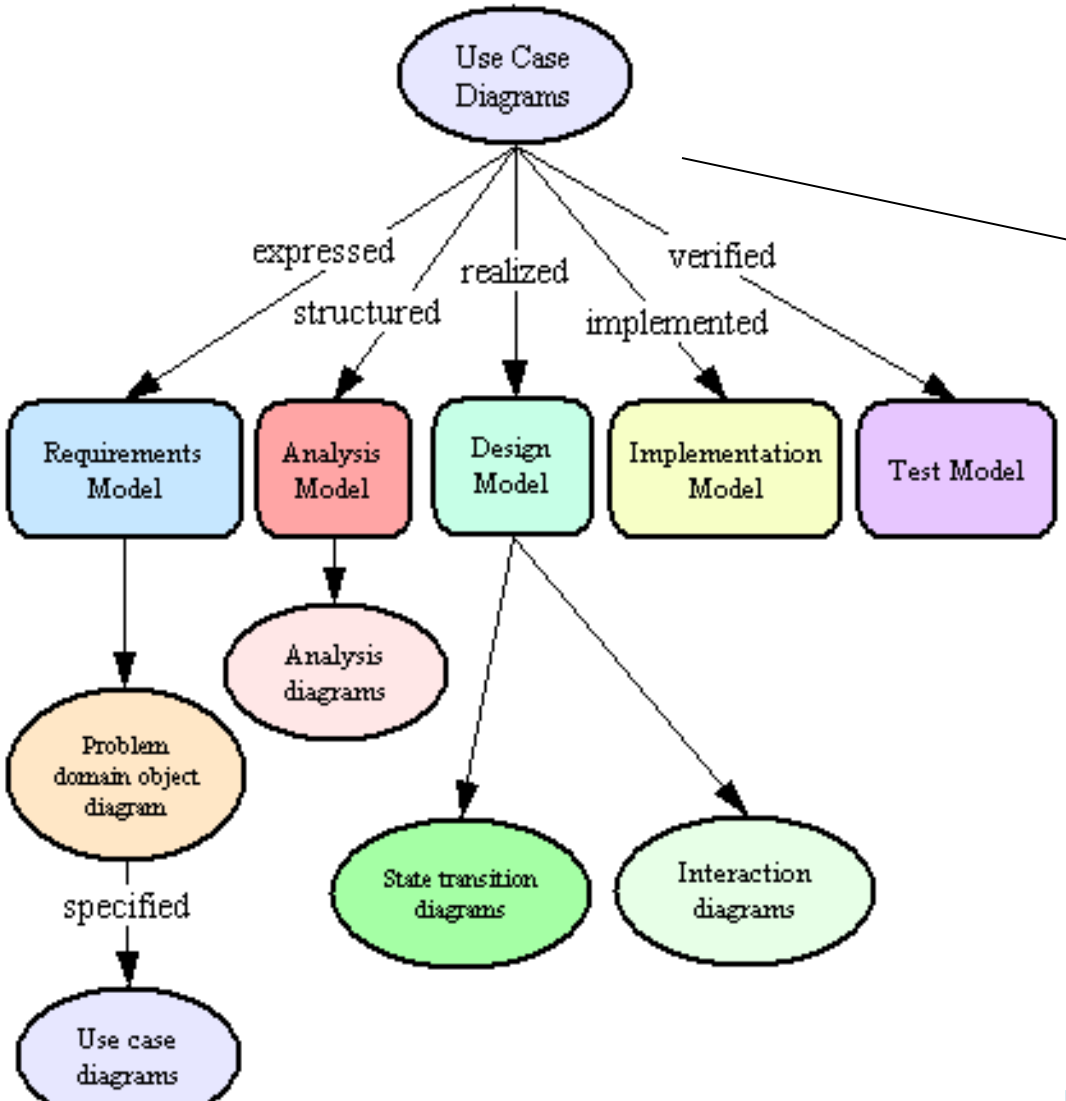
Graphic Languages 7

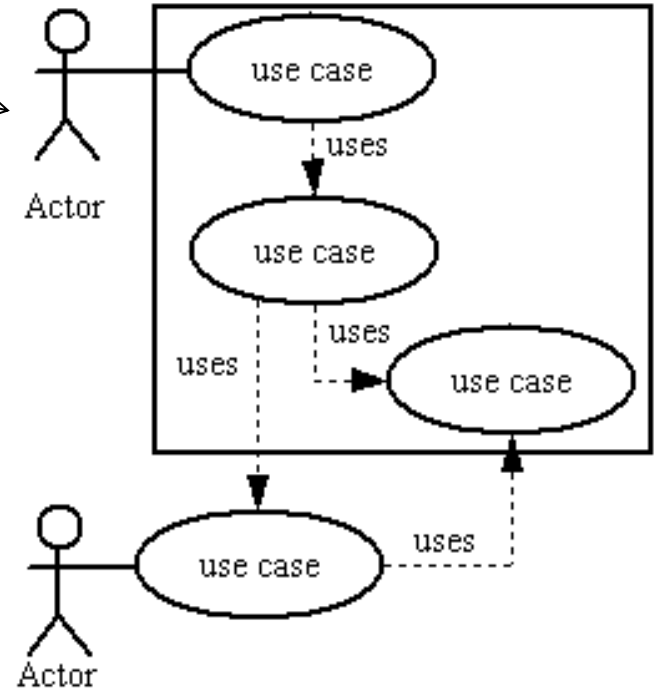
- OMT (object-modeling technique)



Graphic Languages 8

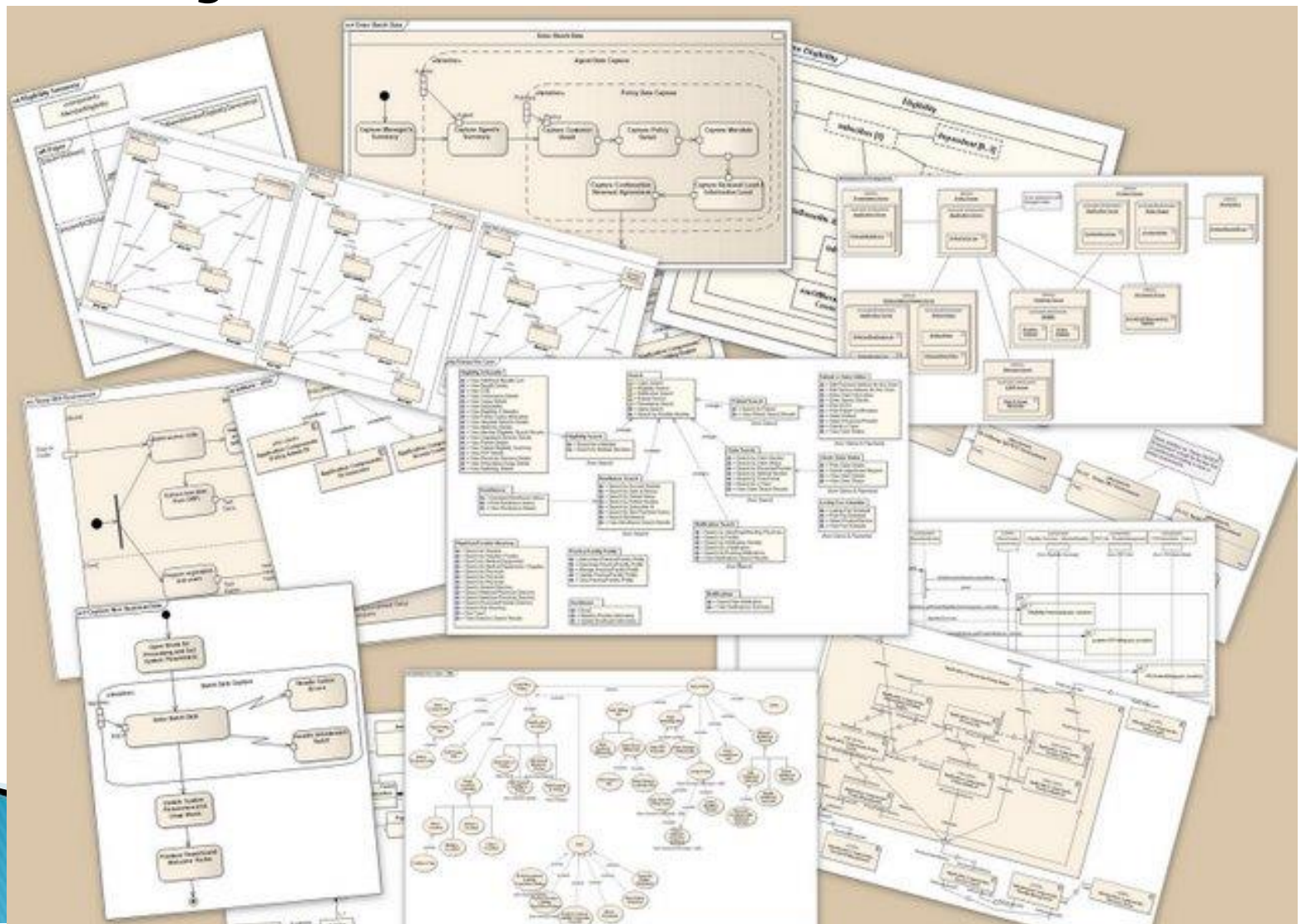
- OOSE (Object-oriented software engineering)





Graphic Languages 9

- UML Diagrams

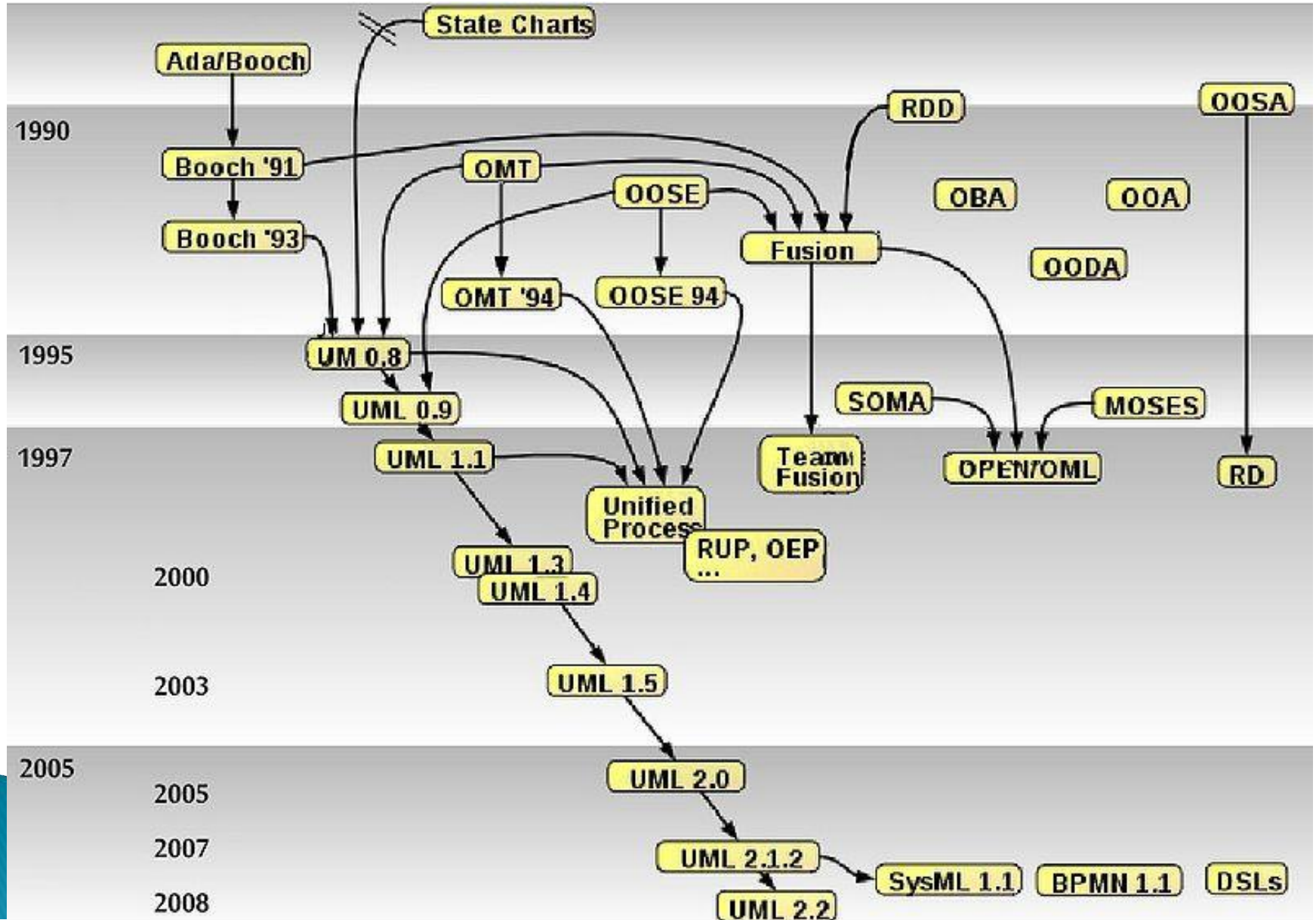


UML – Introduction

- UML (Unified Modeling Language) – it is the successor of the three best OO modeling languages:
  - Booch (Grady Booch)
  - OMT (Ivar Jacobson)
  - OOSE (James Rumbaugh)

- UML consists of the union of these modeling languages and in addition it has a greater expressiveness

UML – Evolution

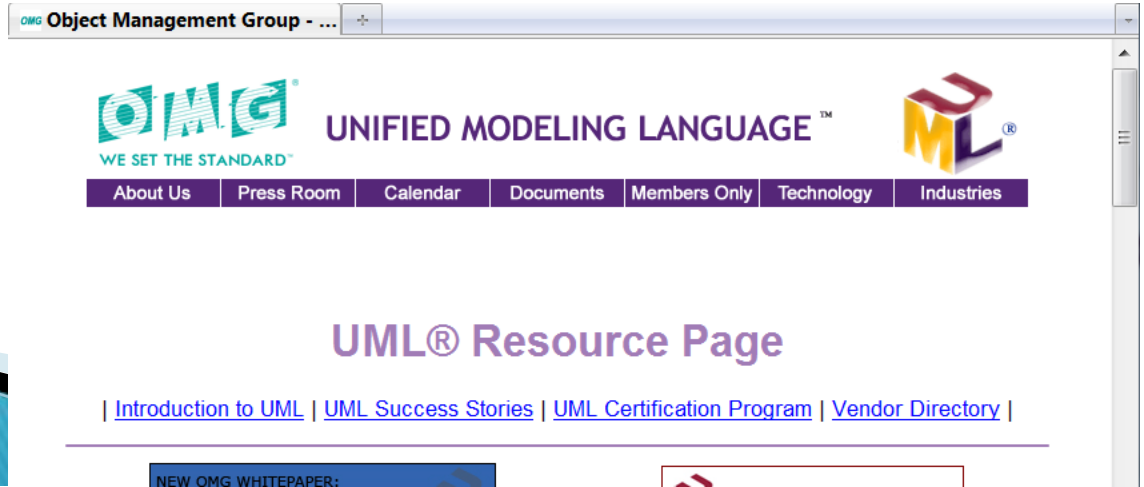


UML – Definition (OMG)

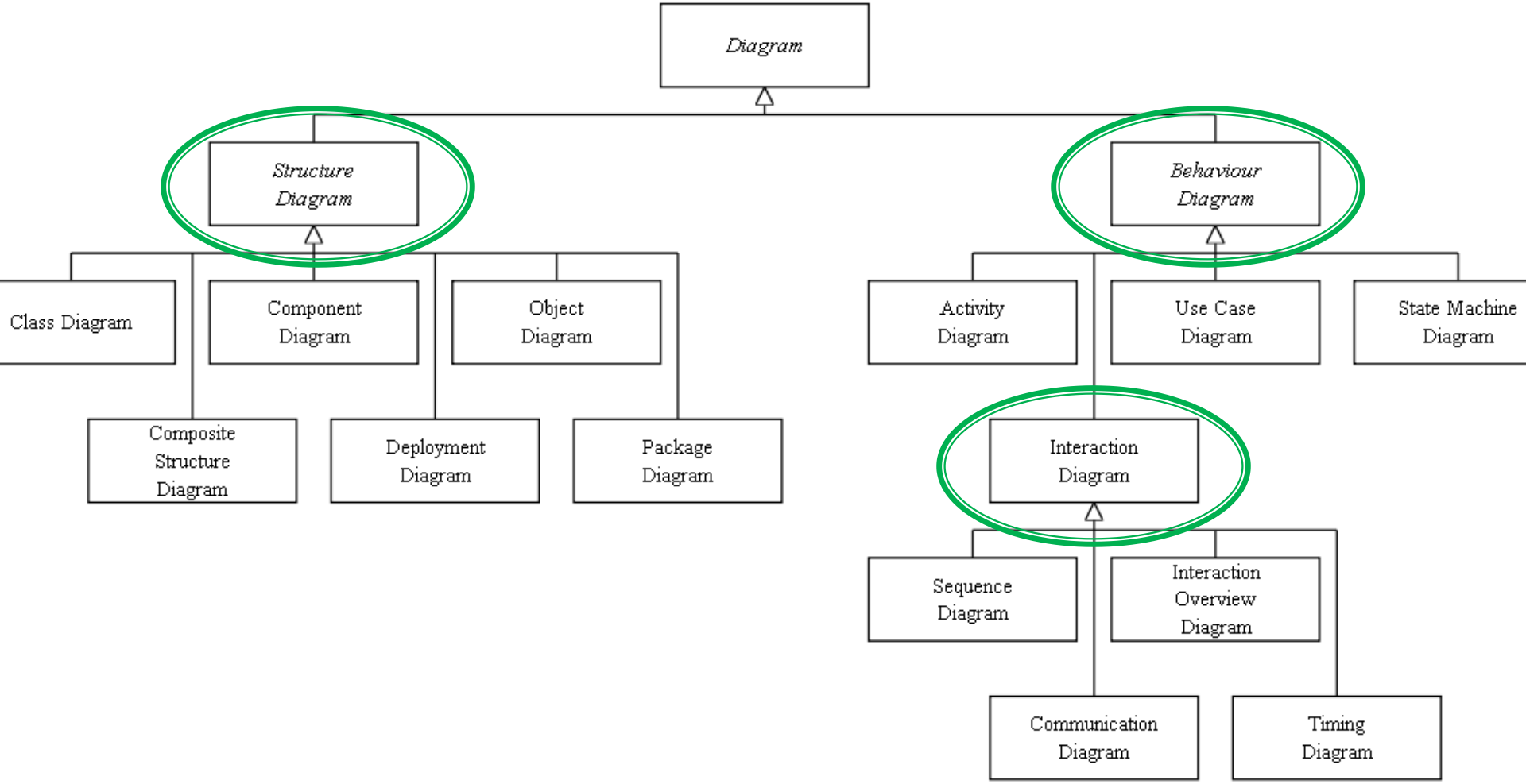
- *"The Unified Modeling Language (UML) is a* ***graphical language for visualizing, specifying, constructing, and documenting the artifacts of a software-intensive system.***
- *The UML offers a* ***standard way to write a system's blueprints****, including conceptual things such as* ***business processes*** *and* ***system functions*** *as well as concrete things such as* ***programming language statements****,* ***database schemas****, and* ***reusable software components****."*

UML – International Standard

- January 1997 – UML 1.0 was proposed for standardization by OMG (Object Management Group)
- November 1997 – Version UML 1.1 was adopted like standard by OMG
- Last version is UML 2.5.1 (December 2017)
- Official site: http://www.uml.org

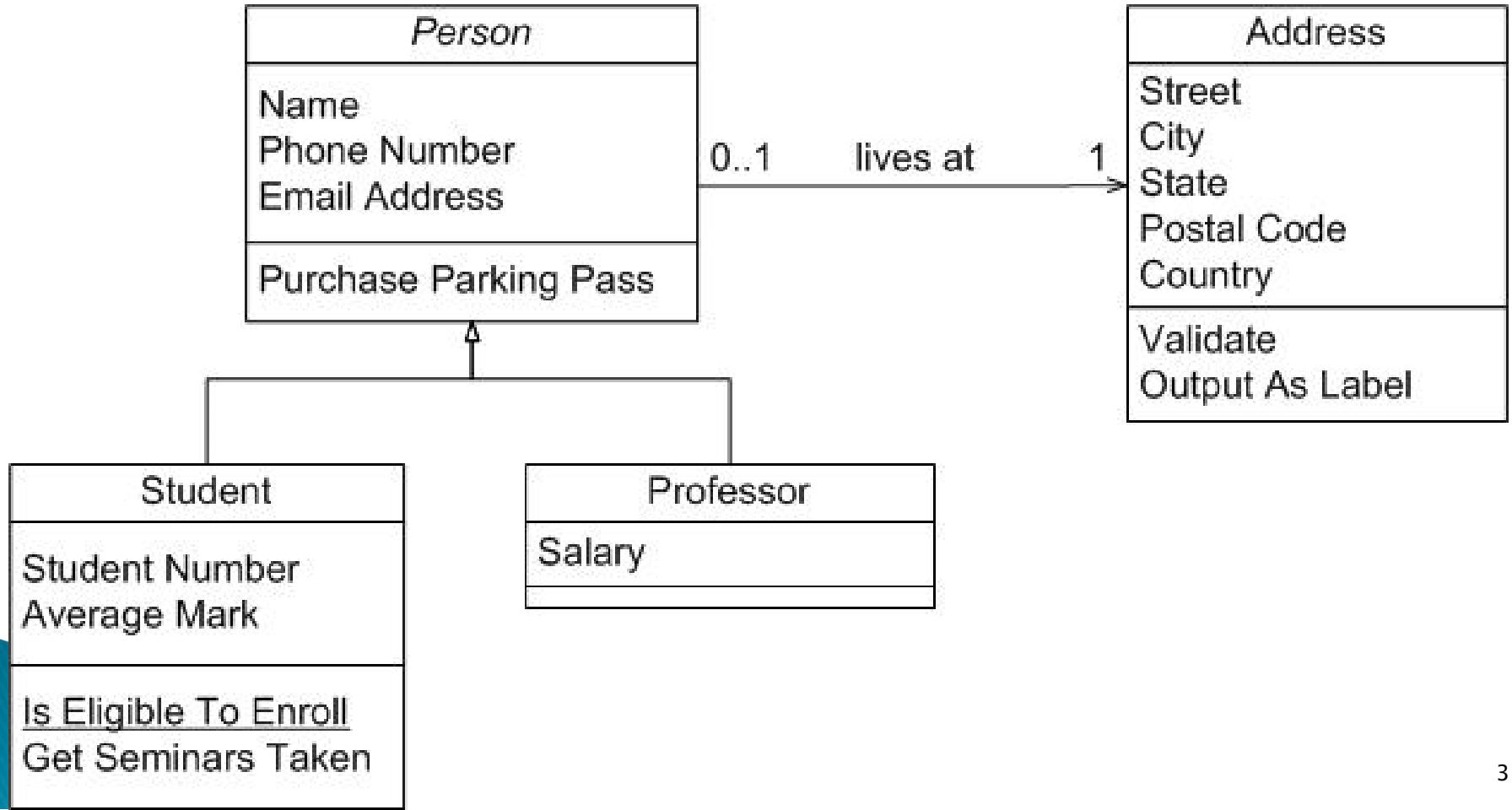


UML2.0 – 13 Types of Diagrams



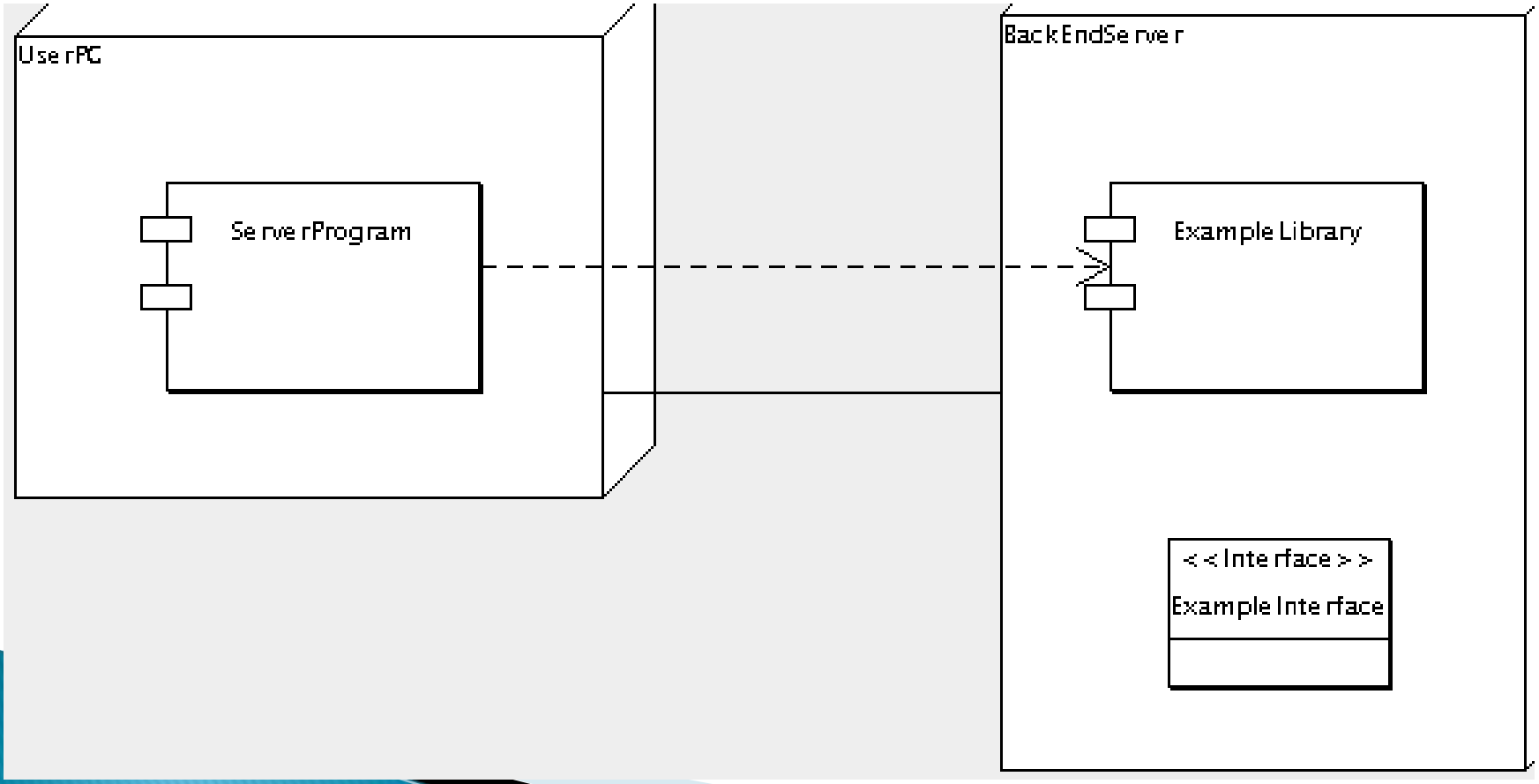
UML2.0 – Structure Diagrams 1

- **Class Diagrams**: classes (attributes, methods) and relations between classes



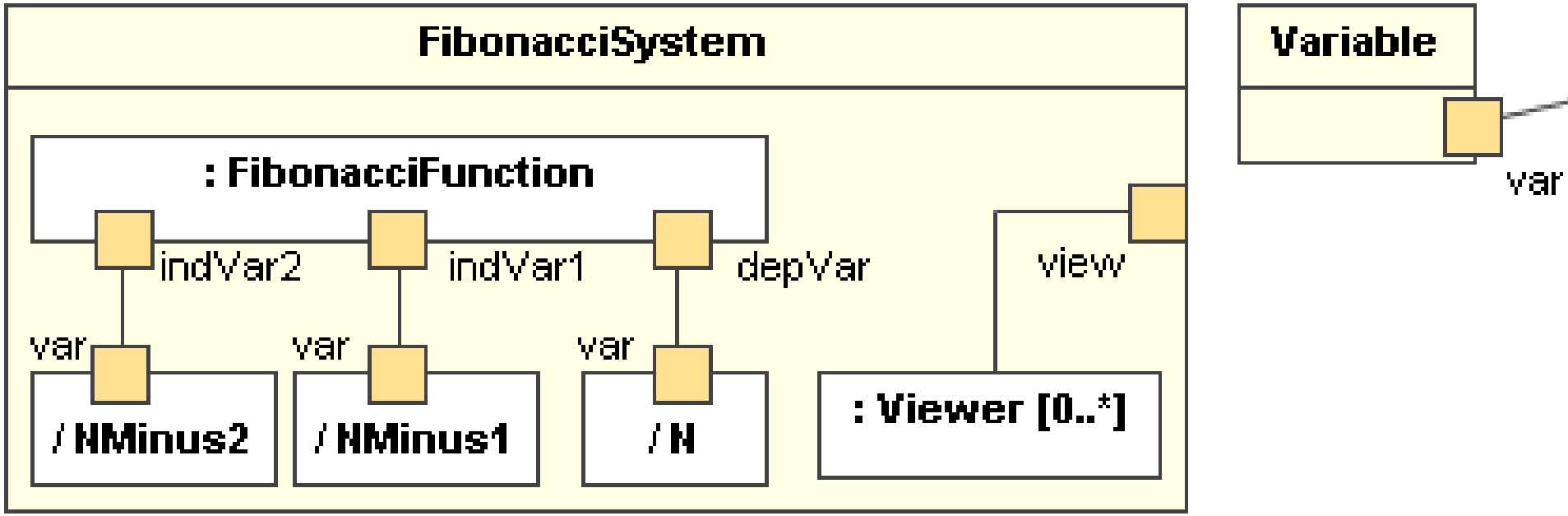
UML2.0 – Structure Diagrams 2

- **Component Diagrams:** main components and relations between them



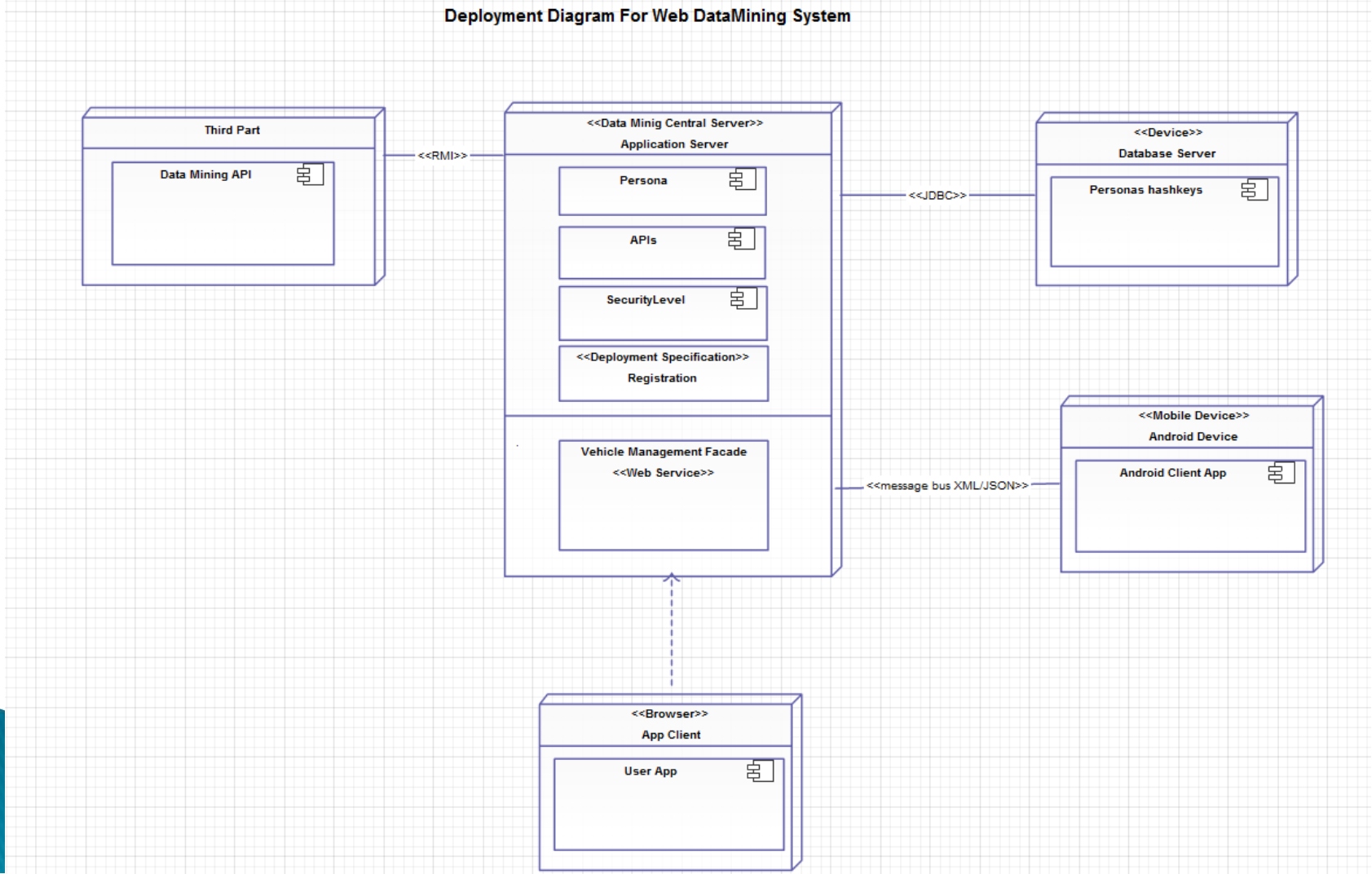
UML2.0 – Structure Diagrams 3

- **Composite structure diagrams**: intern structure



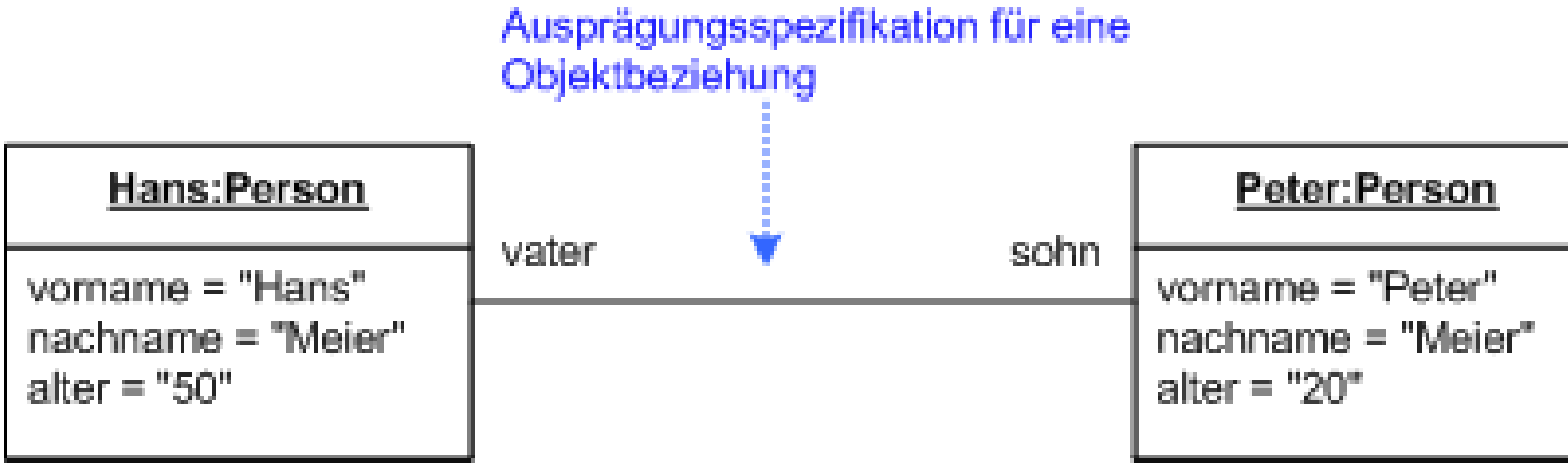
UML2.0 – Structure Diagrams 4

- **Deployment Diagram**: hardware structure



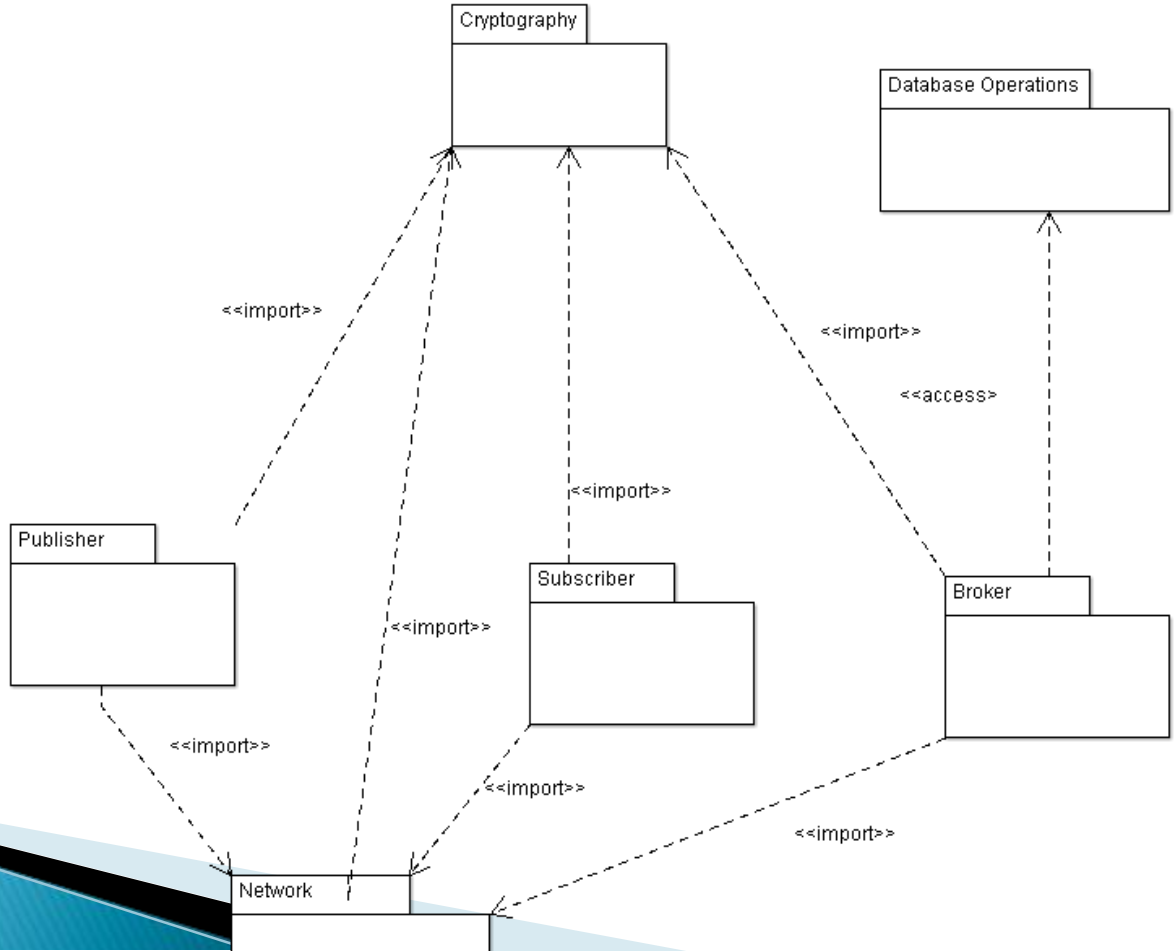
UML2.0 – Structure Diagrams 5

- **Object Diagrams**: system structure at a given time



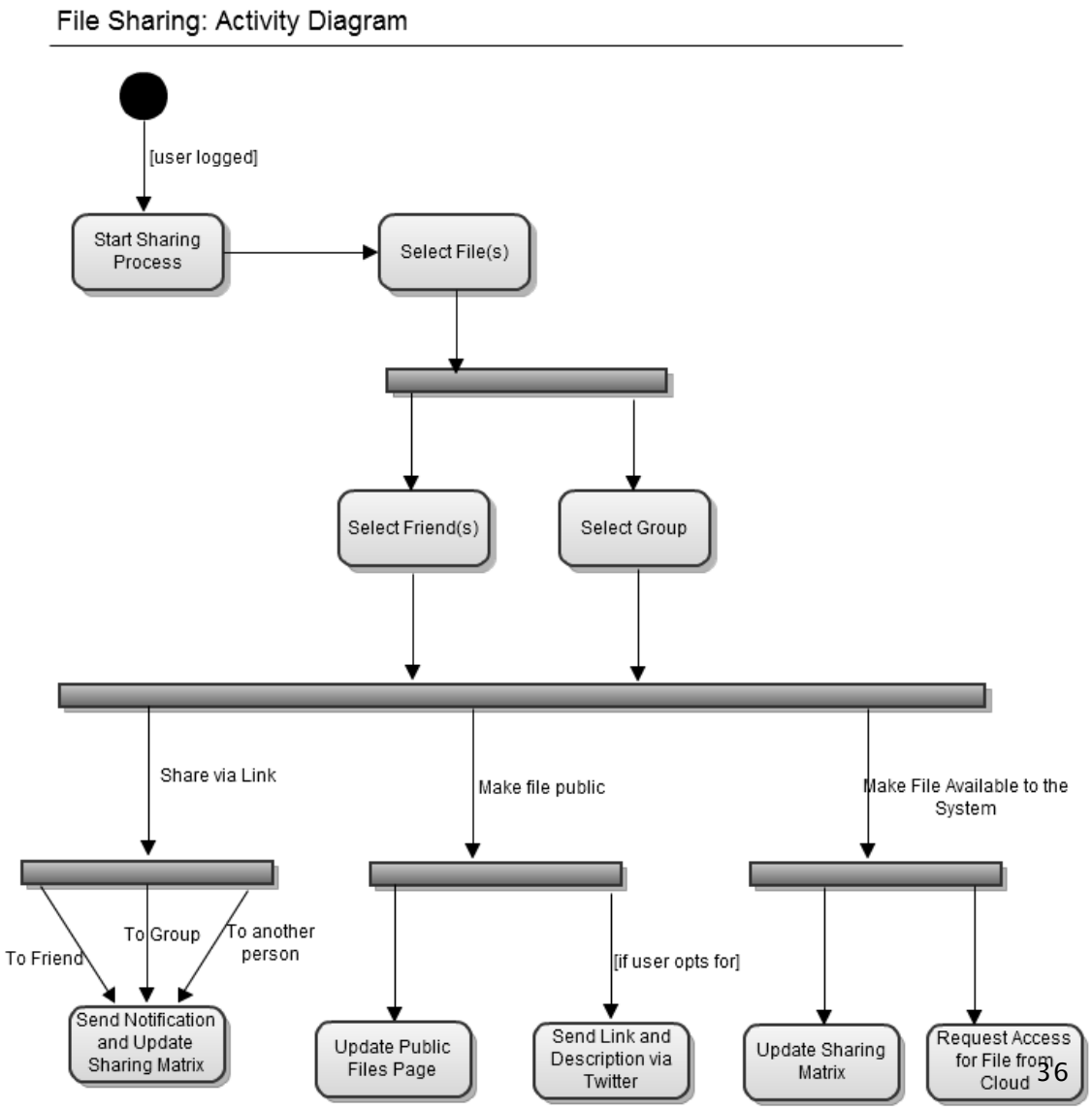
UML2.0 – Structure Diagrams 6

- **Diagram package:** the system is divided into packages with the relationships between them



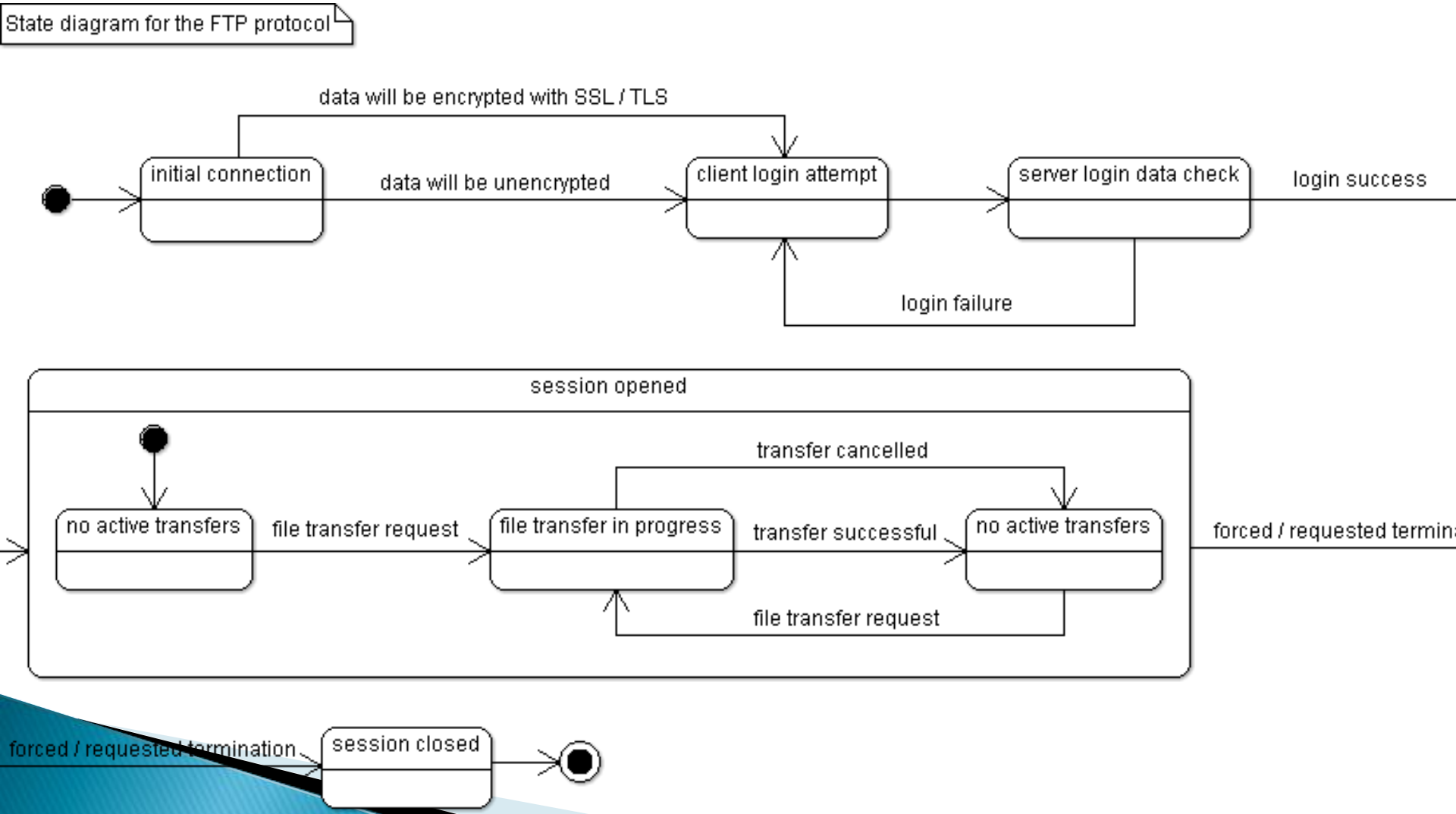
UML 2.0 – Behavioral Diagrams 1

- **Activity diagrams:** business presentation and workflow



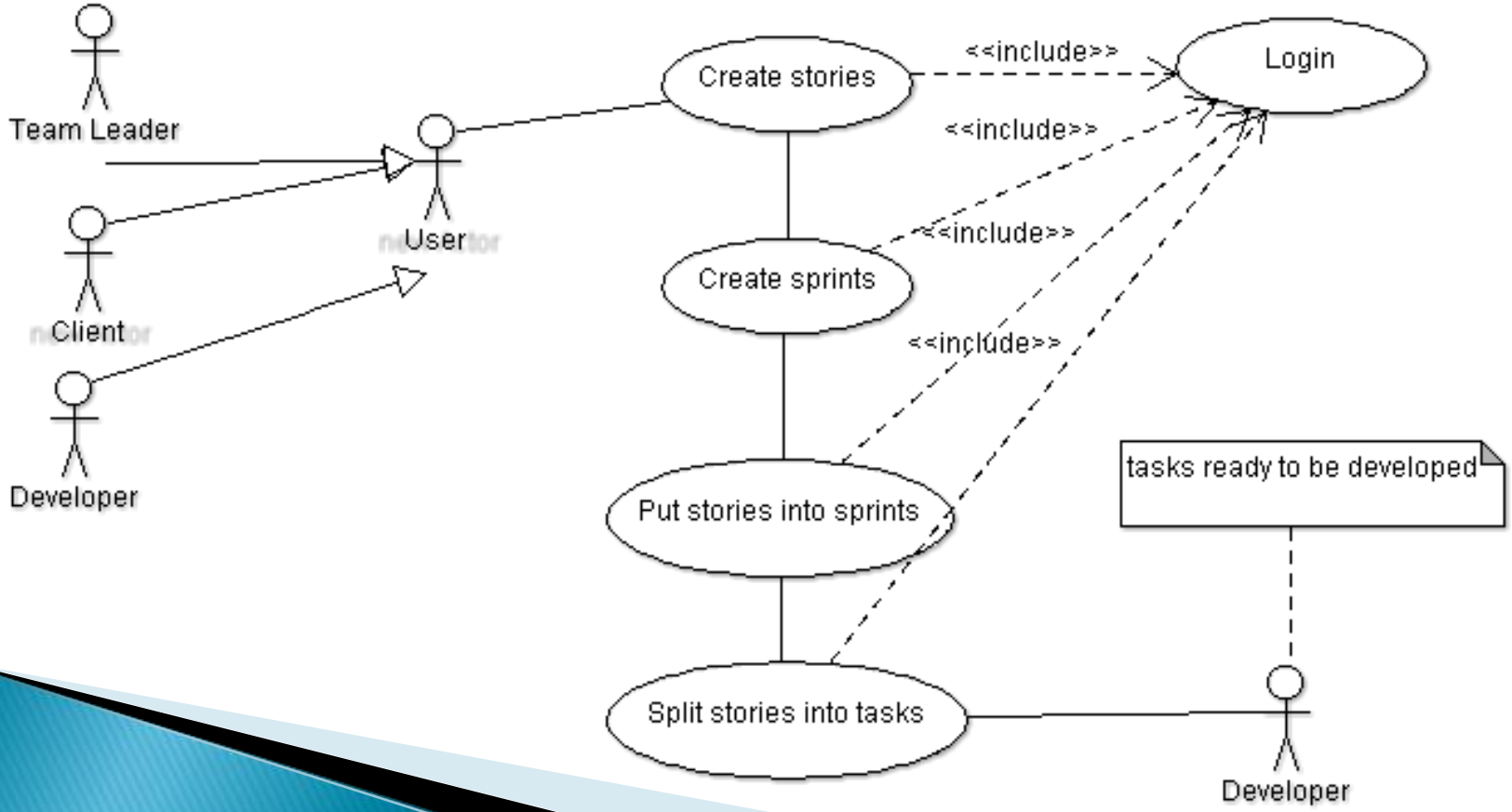
UML 2.0 – Behavioral Diagrams 2

- **State diagrams:** to present states of objects



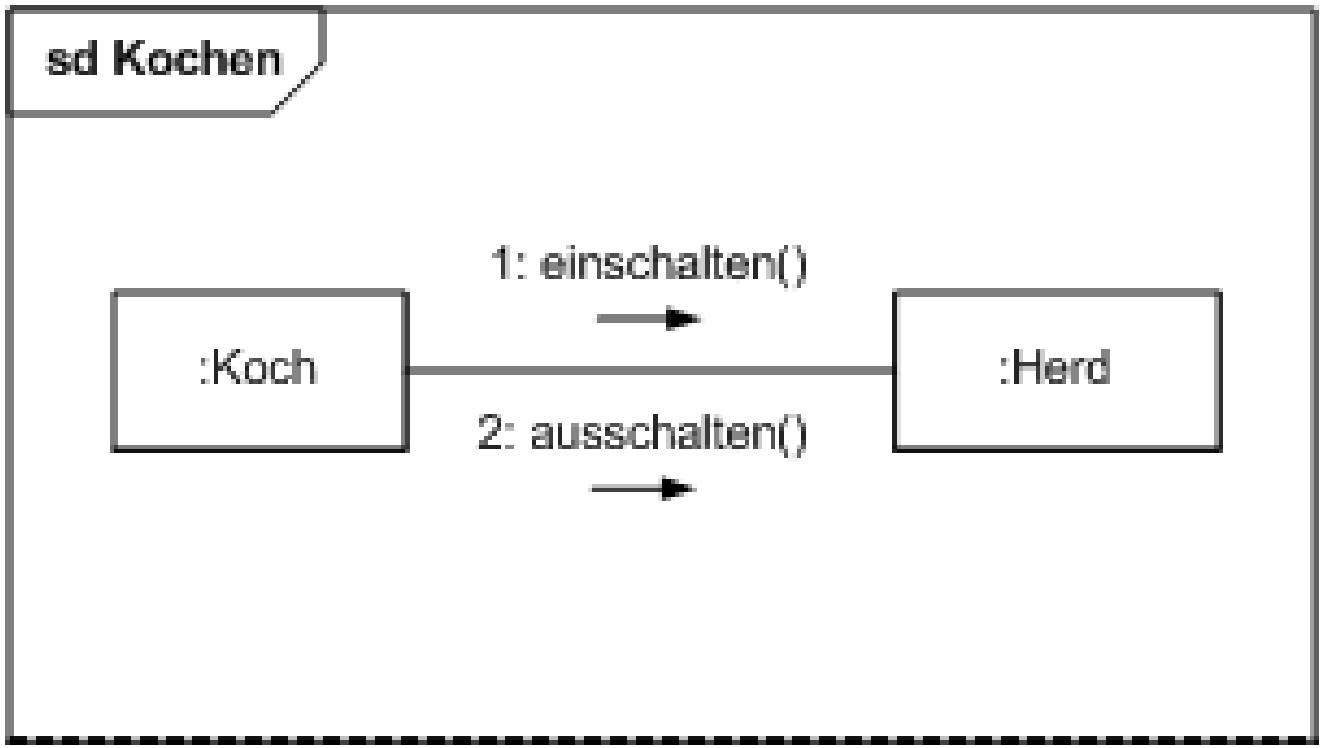
UML 2.0 – Behavioral Diagrams 3

- **Use Case Diagrams**: show the functionality of the system using actors, use cases, and dependencies between them



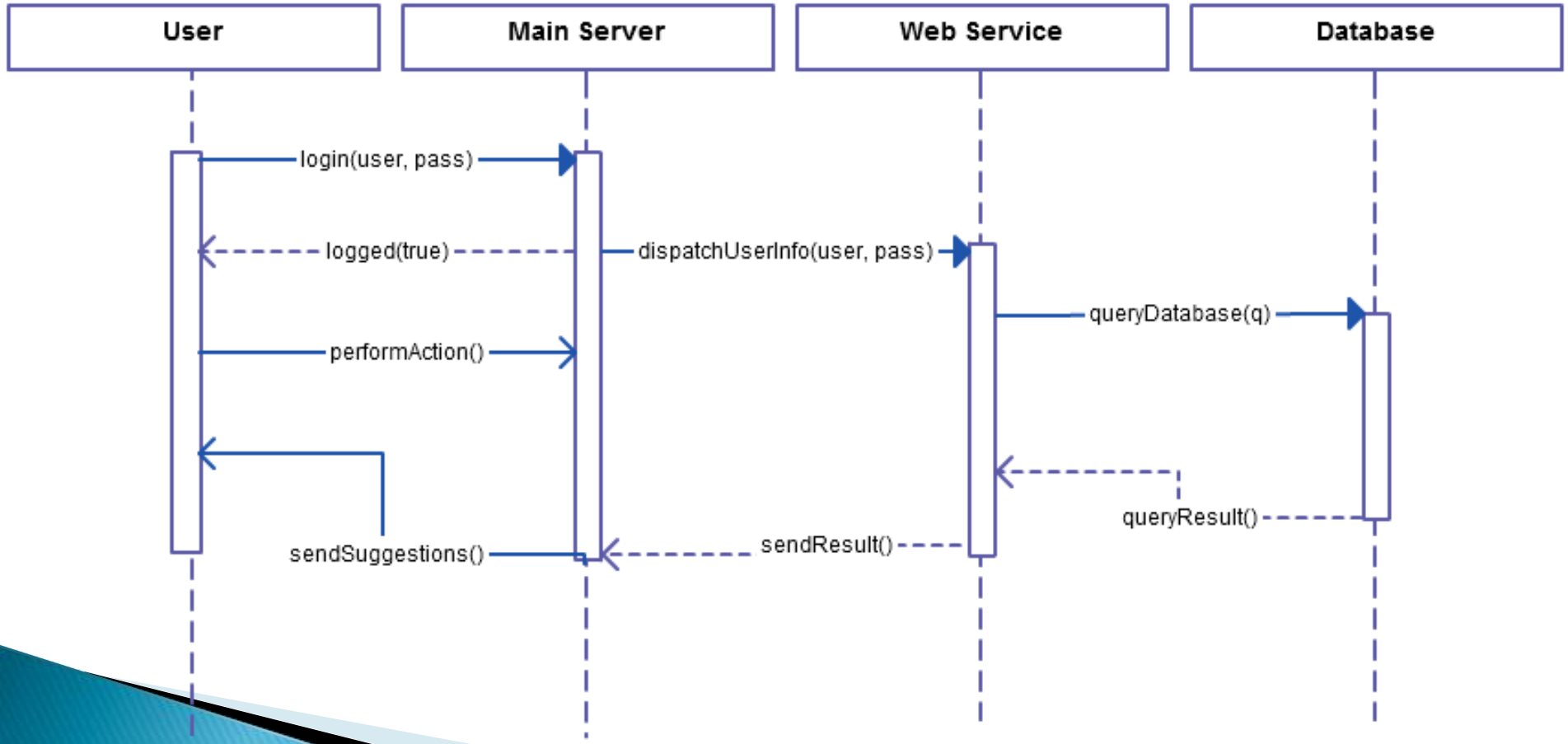
UML 2.0 – Interaction Diagrams 1

- **Communication diagram:** shows the interactions between objects (the dynamic behavior of the system) (actors: chef, stove, actions: cooking, firing, disconnection)



UML 2.0 – Interaction Diagrams 2

- **Sequence Diagram:** shows how objects communicate with each other in terms of sending messages



[online diagramming & design] creately.com

Class Diagram

- **Purpose:**
  - Modeled vocabulary of the system to be developed
  - Capture semantic connections and interactions that are established between components
  - Used to model the structure of a program
- **Contains**
  - Classes/Interfaces
  - Objects
  - Relations (Association, Aggregation, Generalization, Dependence)

Classes

- Modeled vocabulary = identifies the concepts that the client or the programmer uses to describe the problem solution
- Elements of classes:
  - Name: identifies a class
  - Attributes: properties of the class
  - Methods: implementing a service that can be called by any instance of the class

Name

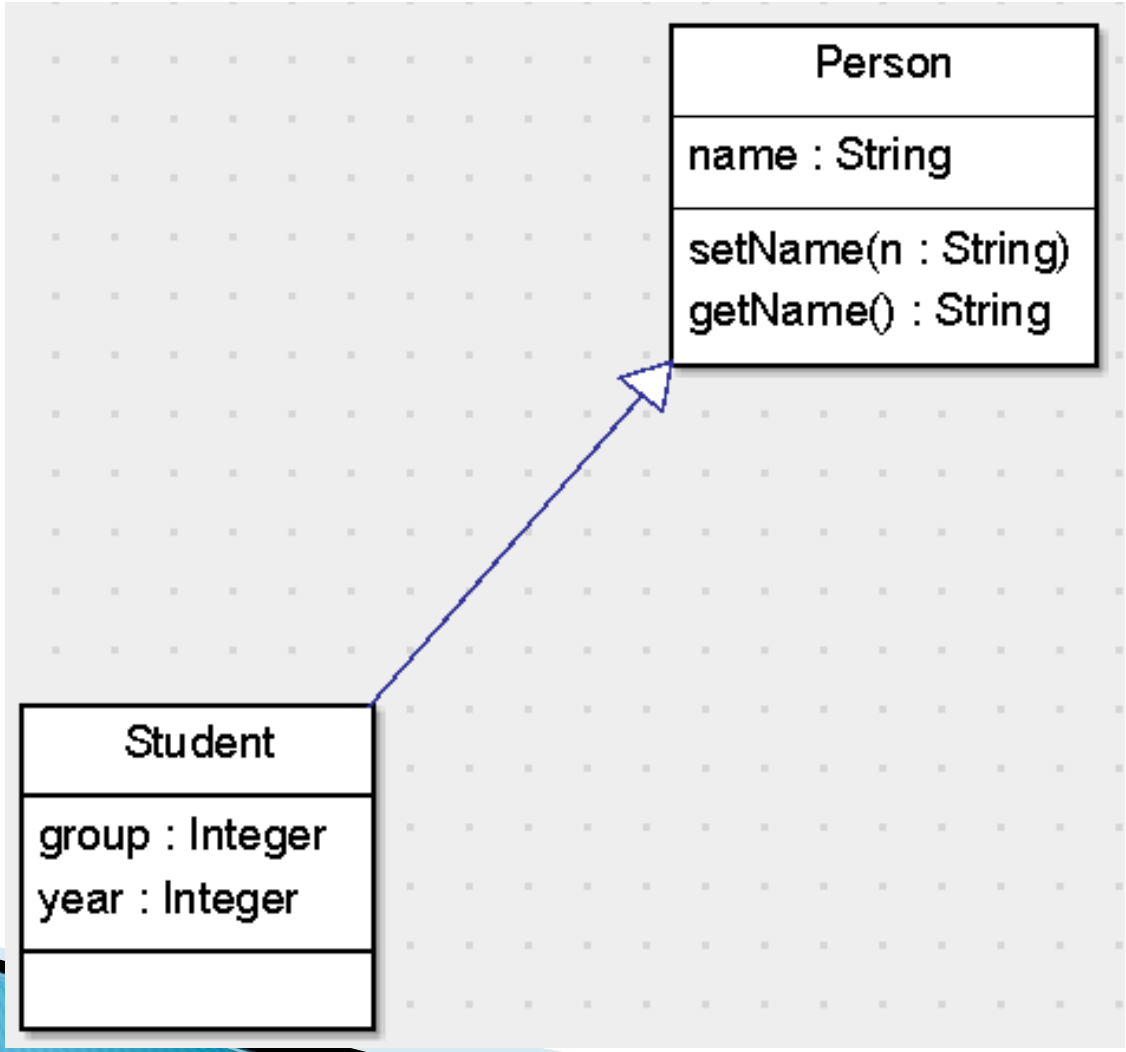
Attributes (private)

Methods (private, protected)

Relations – Generalization – C#

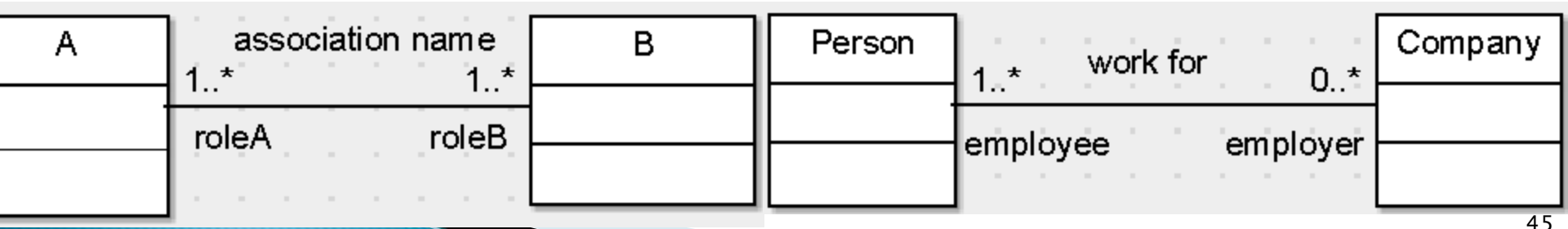
- Modeled concept of inheritance between classes
- Also called the relationship of type "is a"

ArgoUML – Generalization



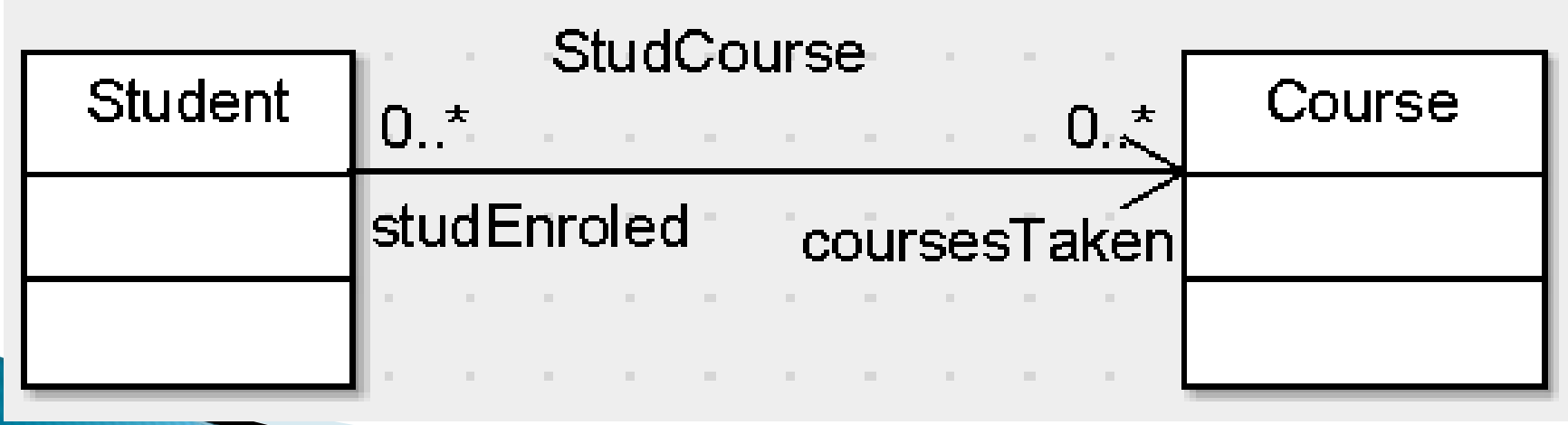
Association

- Expresses a semantic connection or interaction between objects belonging to different classes
- As the system evolves new connections between objects can be created, or existing connections can be destroyed
- An association interact with objects through its association heads
- Elements:
  - Name: describe the relation
  - Heads of association
    - Name = role of the object in relation
    - Multiplicity = how many instances of a class correspond to a single instance of the other class



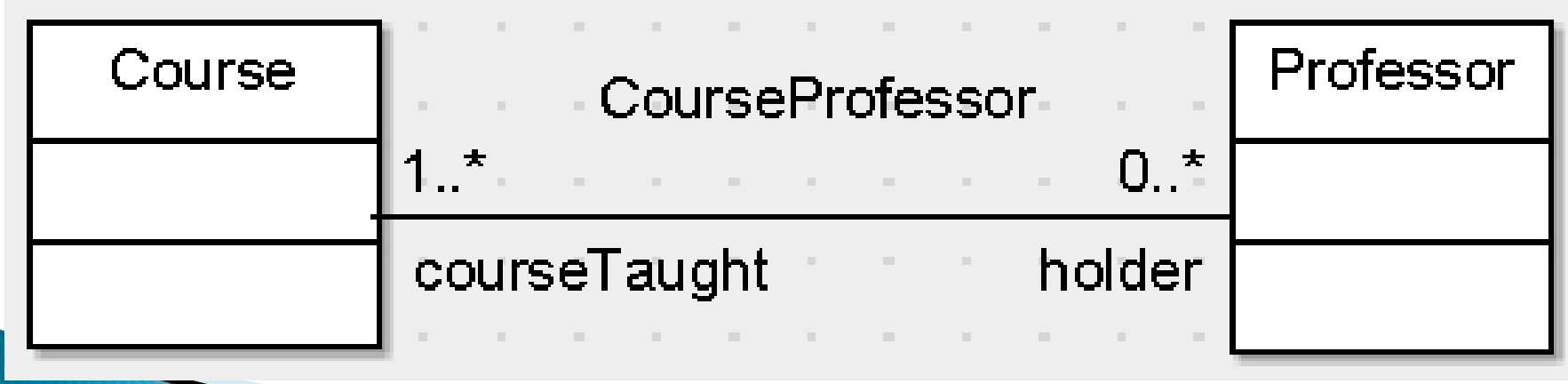
Association 1

- The relation Student – Course
  - **Student**: follows 0 or more courses, courses know that you follow;
  - **Course**: it can be followed by several students, it does not know students who follow it



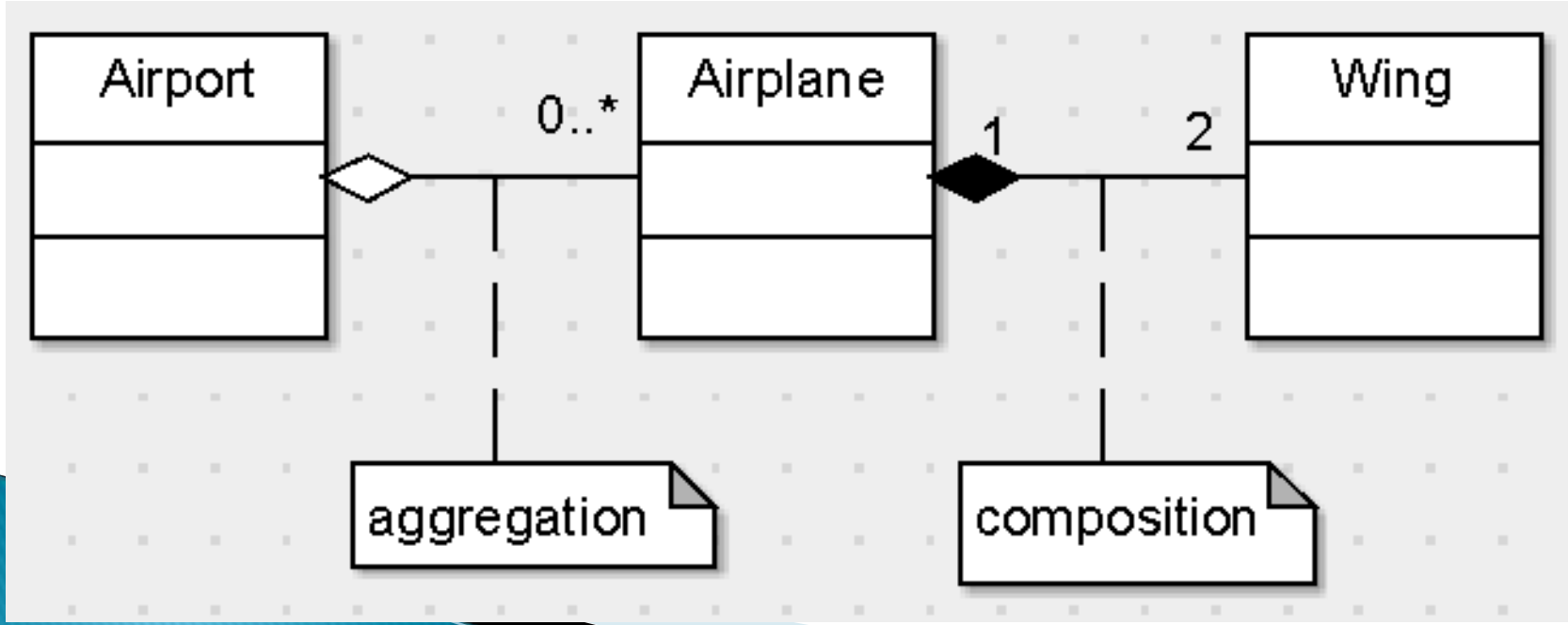
Association 2

- Relation Course – Professor
  - **Course**: I am taught by a teacher, I know the holder
  - **Profesor**: I can teach many courses, I know the courses they teach

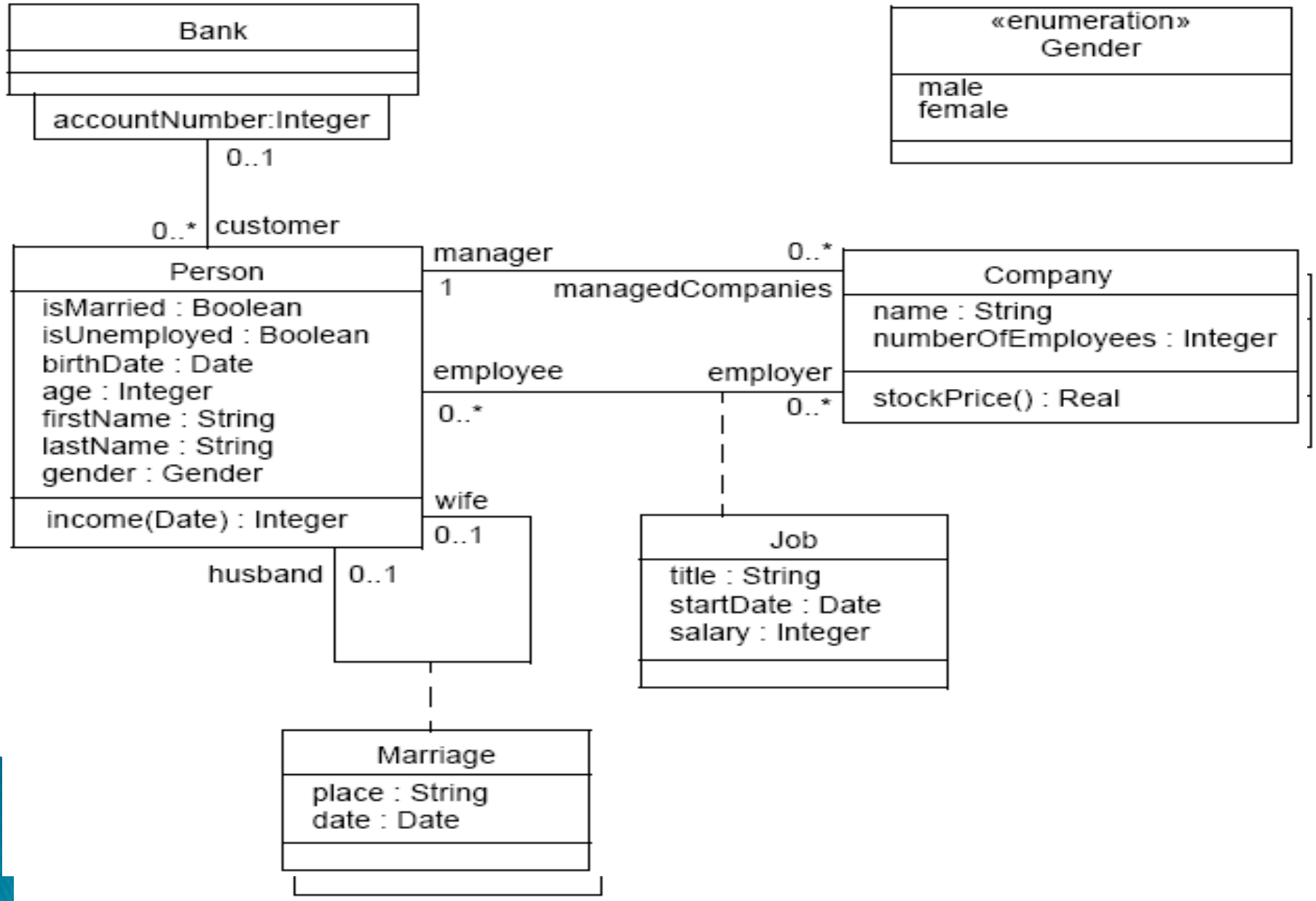


Aggregation

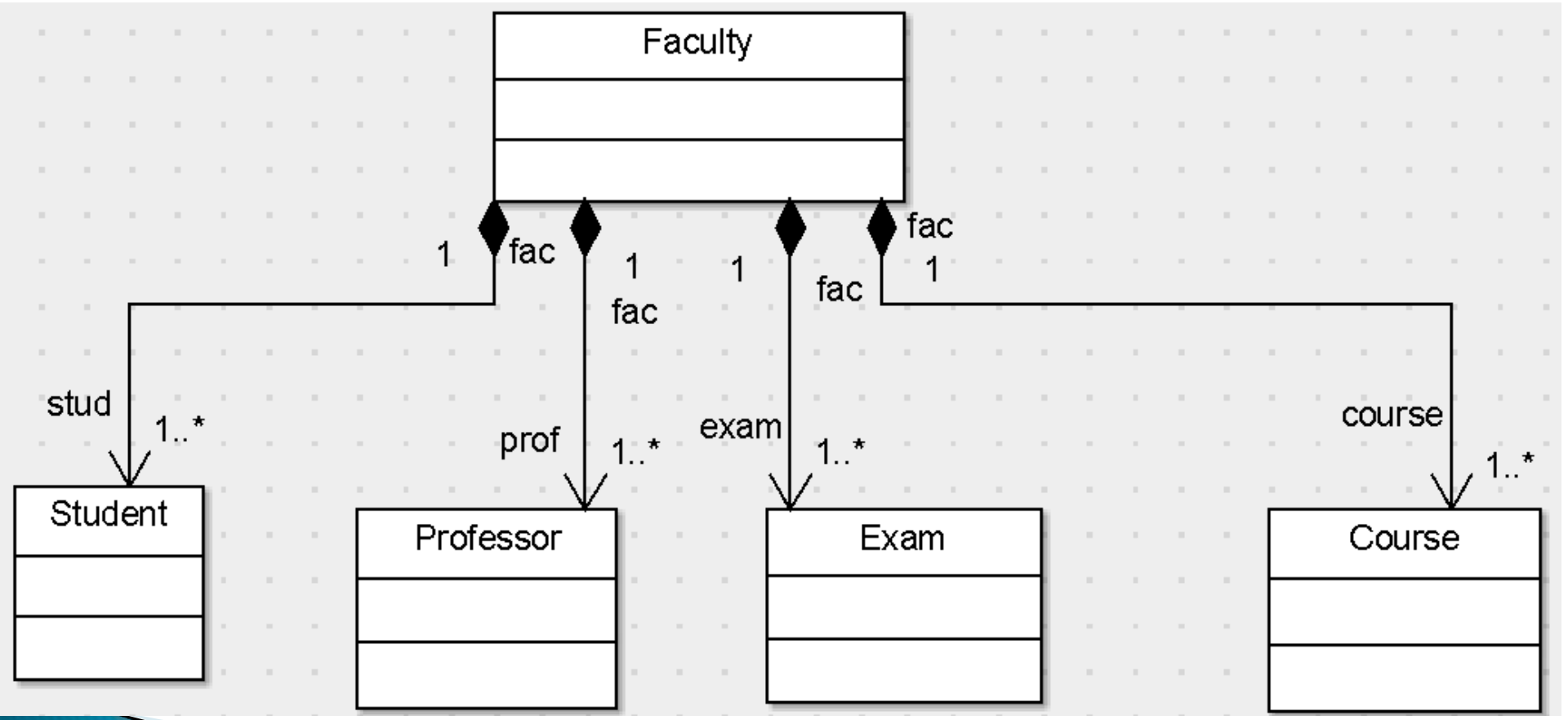
- It is a particular case of the association relation
- It models a part-whole relationship type
- It can have all the elements of partnership agreements, but in general it only specifies the multiplicity
- It is used to model situations in which an object is made up of several components



Example


Bank
accountNumber:Integer
0..1
0..*
customer
«enumeration»
Gender
male
female
Person
isMarried : Boolean
isUnemployed : Boolean
birthDate : Date
age : Integer
firstName : String
lastName : String
gender : Gender
income(Date) : Integer
manager
1
managedCompanies
0..*
Company
name : String
numberOfEmployees : Integer
stockPrice() : Real
employee
0..*
employer
0..*
wife
0..1
husband
0..1
Job
title : String
startDate : Date
salary : Integer
Marriage
place : String
date : Date

Composition Relation ("hasA")



Case Study

- Get students with scholarships
  - Actors
  - Use Cases
  - Classes

Conclusions

- Modeling – Why?
- Graphic Languages
- UML
  - Structural: classes
  - Behavioral: use-case
  - Interactions

Questions

- 1) Give an example of a situation in which it is sufficient only a chart or a diagram to do something.
- 2) Give an example of a situation where you need more than a chart or a diagram to do something.
- 3) What is the link between use-case diagrams and class diagrams?

Bibliography

- OMG Unified Modeling Language**TM (OMG UML), Infrastructure,** Version 2.2, May 2008, http://www.omg.org/docs/ptc/08-05-04.pdf
- **ArgoUML User Manual,** A tutorial and reference description, http://argouml-stats.tigris.org/documentation/printablehtml/manual/argomanual.html
- Ovidiu Gheorghieş, Curs IP, Courses 3, 4
- UML Diagrams, Regie.ro

Links

- OOSE: http://cs-exhibitions.uni-klu.ac.at/index.php?id=448
- ArgoUML: http://argouml-stats.tigris.org/nonav/documentation/manual-0.22/
- Wikipedia